

A research-grade catalog of fast matrix multiplication algorithms with frontier-closure search*

Benoit Lacelle
benoit@solven.eu

Draft, June 10, 2026

Abstract

The 2022–2026 burst of activity in small-format matrix multiplication (AlphaTensor 2022, AlphaEvolve 2025, Schwartz–Zwecher 2025) has produced striking individual results but scattered them across different fields, attribution conventions, and serialisation formats. A complementary line of work — Perminov’s open-source flip-graph framework [16, 15] — instead drives existing construction methods, notably flip-graph and *meta-flip-graph* search, at scale across large format spaces, discovering many new low-rank schemes (including ternary-integer ones) that further enrich the landscape this catalog must unify. We present a unified, machine-checkable catalog covering shapes up to $\langle 32, 32, 32 \rangle$ over $\mathbb{Q}$, $\mathbb{Z}$, $\mathbb{R}$, $\mathbb{C}$, and $\mathbb{F}_2$, with a separate axis for commutative algorithms (Waksman 1970, Makarov 1986, Rosowski 2019).

Derivation over this catalog is performed by a *frontier-closure search* that recombines catalog entries by axis-flip, Kronecker, axis concatenation, serendipitous products, recombination-with-allocation (with optional output peeling and pair fusion), and downward projection.

A central methodological point is the *non-overlap property*: our recombination does not, and cannot, rediscover the shared bilinear products that hand-crafted constructions (Strassen, Laderman, Smirnov, AlphaTensor) are built around. This draws a clean line between the “find a cleverer bilinear core” and “compose known cores” axes of progress, and resolves several attribution puzzles in the literature.

We refresh the DIS09 comparison tables, split per field and with a commutative column, and provide the tooling to regenerate them automatically as the catalog evolves.

1 Introduction

The complexity of multiplying two $n \times n$ matrices has been studied since Strassen’s 1969 algorithm [23] broke the cubic barrier, and remains open: the exact value of the matrix multiplication exponent ω is unknown, though sub-cubic upper bounds have been driven down to $\omega < 2.371 \dots$ via the laser method and its successors. The asymptotic regime has been productive for theoretical upper bounds but has produced no concrete algorithms one could implement for finite n ; the constants are astronomical.

Meanwhile, the *small-format* regime – exact rank bounds for fixed $\langle n, m, p \rangle$ tensors with $n, m, p \leq 32$ or so – has lived a quieter life dominated by hand-constructed schemes: Strassen (1969) [23], Hopcroft–Kerr (1971) [6], Laderman (1976) [9], Pan (1978, 1980) [12, 13], and Smirnov (1986, 2013) [21, 22]. These results are concrete algorithms; they multiply small matrices using a count of bilinear products that beats the naïve nmp . When applied recursively, each small-format rank improvement compounds into an ω improvement. The strongest *practical* small-format exponent comes from Smirnov’s $\langle 3, 3, 6 \rangle$ scheme of rank 40 [22]: at $\omega = 3 \log 40 / \log 54 \approx 2.7743$

*Source repository: github.com/solven-eu/matmulcatalog. This article is generated from `paper/article.tex` alongside the catalog; numerical tables track the catalog by construction.

it sits below the exponent of every explicit cubic scheme in the catalog, making it the best base case a recursive implementation can realistically use.

The 2022–2026 period has shaken this landscape. AlphaTensor [5] found 47 bilinear products for $\mathbb{F}_2\langle 4, 4, 4 \rangle$ – previously believed to require 49. AlphaEvolve [11] found 48 for $\mathbb{C}\langle 4, 4, 4 \rangle$, which Dumas–Pernet–Sedoglavic [4] promptly rationalised — 48 with coefficients in $\{\pm 1, \pm \frac{1}{2}, \pm \frac{1}{4}, \pm \frac{1}{8}\}$, valid over $\mathbb{Q}$ and hence $\mathbb{R}$. Schwartz–Zwecher 2025 [19] produced a systematic disjoint-sum construction giving record ranks at several large shapes. The FMM catalog at Lille [20] and Perminov’s `FastMatrixMultiplication` repository [14] aggregate schemes from across the literature with their own derivations layered on top.

The problem. These results live in incompatible formats (Maple scripts, NumPy archives, ad-hoc JSON), use inconsistent shape conventions (sorted vs unsorted $\langle n, m, p \rangle$, with or without explicit field tags), and carry inconsistent attribution. A $\langle 4, 4, 4 \rangle$ entry sourced from AlphaTensor is sometimes cited as “the AlphaTensor scheme” even when the rank was already established by Strassen recursion or Smirnov – only some of AlphaTensor’s per-format ranks were genuine improvements over prior SOTA. The same trap exists in every bulk-import flow we have audited.

We argue that a research-grade catalog – machine-checkable, fully attributed, and uniformly serialised – is now both possible and necessary. This paper describes the construction of such a catalog, together with a frontier-closure search that automatically discovers recombinations of catalog entries.

Field discipline as load-bearing. A bare $\langle n, m, p \rangle$ is ambiguous: different fields admit different ranks. The canonical example is $\langle 4, 4, 4 \rangle$: 47 over $\mathbb{F}_2$ (AlphaTensor), 48 over $\mathbb{C}$ and — since the Dumas–Pernet–Sedoglavic rationalisation [4] — over $\mathbb{Q}$ and $\mathbb{R}$ too, while the best *integer-coefficient* scheme remains 49 (Strassen squared). Between AlphaEvolve and [4] the landscape was sharper still, with $\mathbb{R}$ standing at 49: a field-tag-free “ $\langle 4, 4, 4 \rangle = 48$ ” would have been wrong over $\mathbb{R}$ for those weeks, and is still wrong over $\mathbb{Z}$ and $\mathbb{F}_2$ today (where 47 beats it). We refuse to merge across fields and require every rank claim in the catalog (and in this paper) to carry a field tag. Commutative algorithms (Waksman 1970 [24], Makarov 1986 [10], Rosowski 2019 [17]) form a separate axis: they are valid for scalar matrix multiplication but do *not* lift to recursive multiplication over non-commutative rings.

What the catalog catalogs. Each entry encodes either

- (a) explicit factor matrices (U, V, W) with field tag, addition count, and lineage record; or
- (b) a *cited bound* – a rank claim attributed to a source whose explicit factor matrices we do not (yet) hold; or
- (c) a *derived bound* – a rank computed by a formula that our code can re-derive on demand (Waksman, Rosowski, Pan trilinear aggregation).

This three-way split lets us include results we cannot yet materialise as schemes without misrepresenting their status, and lets us regenerate the derived layer as our formulas improve.

Derivation over the catalog. Recursive multiplication over the same catalog is performed by a search that recombines existing entries via well-defined operators (Section 5): axis-flip, Kronecker product, axis concatenation, recombination with allocation (with optional output peeling and pair fusion), the serendipitous (bud) product, and downward projection. The search is iterated until no further rank improvements appear at any catalog shape – a *frontier closure* – and the resulting wins are materialised lazily after the search has converged. The Schönhage τ -theorem *disjoint-sum* constructions (a target tensor written as a sum of smaller matmuls on disjoint index blocks, whose ranks add) are a different mechanism: we currently *import* such schemes rather than generate them, and a disjoint-sum search operator is the largest open extension (Section 11).

The non-overlap property. An observation that we believe is underdiscussed: our composition operators cannot produce sharing across bilinear products that did not already exist in the source schemes. A scheme assembled by recombination has rank $\sum_k r_{\text{sub-}k}$ on the nose; there is no mechanism inside our materialisation step to discover a new shared core. Hand-crafted schemes work differently – their entire *raison d’être* is to find non-obvious sharing. We argue that this makes “composed scheme matches hand-crafted scheme’s rank” a meaningful co-existence claim, not a re-discovery claim. Section 6 develops this.

Outline. Section 3 fixes notation and the field-discipline conventions. Section 4 describes the catalog’s serialisation, lineage records, and the explicit/cited/derived split. Section 5 catalogs the composition operators. Section 6 develops the non-overlap property. Section 9 describes the frontier-closure search algorithm. Section 10 presents refreshed DIS09-style comparison tables, per field and with a commutative column. Section 11 closes with open questions, including ones the catalog cannot yet answer.

2 Wording

This paper uses the following vocabulary consistently. Variants that appear in the cited literature (“base” vs “atom”, “factorisation” vs “lineage”) are noted in the entries below.

MM. Matrix multiplication. Throughout this paper, an MM *scheme* for shape $\langle n, m, p \rangle$ is a bilinear algorithm (U, V, W) computing $C = A \cdot B$ for $A \in K^{n \times m}, B \in K^{m \times p}, C \in K^{n \times p}$ over an algebra K .

Atom. A primitive MM scheme which cannot be derived from other MM schemes in our framework — it is either imported from upstream (FMM-Lille, Perminov, AlphaTensor, AlphaEvolve), produced by a closed-form constructor (Waksman, Rosowski, Pan TA), or declared as such (Strassen, Laderman, AlphaEvolve $\langle 5, 5, 5 \rangle$). Atoms are the leaves of the lineage DAG. Some literature calls these *base* schemes; we use “atom” because the same scheme can be a base for one search round and a derived result of another.

Lineage. An expression describing in a constructive, replayable way how to build an MM scheme from atoms via the derivation operators (recombination, Kronecker, concat, axis-flip, permutation). Each scheme on disk above MATERIALISE_MAX_DIM (currently 16) is stored as lineage-only — the concrete (U, V, W) can be reconstructed by replaying the lineage given the atoms it references.

Allocation. One way to partition a matrix of shape $\langle n, m, p \rangle$ into blocks compatible with an atom of shape $\langle n', m', p' \rangle$: a per-axis composition $(a_1, a_2, \dots, a_{n'})$ with $\sum a_i = n$ on axis A, and analogously on B and C. The block decomposition is the Cartesian product of the per-axis compositions; the atom is applied at the outer level with sub-products carried by the individual blocks.

Optimisation target. This catalog does *not* aim to minimise additions per scheme. Two schemes of equal rank computing the same $\langle n, m, p \rangle$ (e.g. Strassen-18 vs Strassen–Winograd-15) are equivalent for our purposes — both induce the same recursion structure and the same ω . Addition count is metadata: relevant for scalar matmul and occasionally a cache-behaviour tie-breaker, but not a quality measure. What we *do* minimise is the multiset of distinct sub-shapes $\langle n', m', p' \rangle$ the recursion induces — fewer distinct shapes means a simpler block decomposition, less catalog dependency, and easier verification. Thus $\langle 17, 17, 17 \rangle = 2940$ via three sub-shapes versus FMM-Lille’s $\langle 17, 17, 17 \rangle = 2934$ via two: the FMM recipe is structurally preferred regardless of addition counts.

A caveat on the addition figures themselves: the Strassen–Winograd “15” is the *scheduled* count (after common-subexpression sharing). Flat-counted — every addition in the (U, V, W) forms tallied independently — Winograd is ~ 22 – 24 additions, *more* than Strassen’s 18. The scheduled-vs-flat distinction is non-obvious and is the reason raw addition counts are an unreliable cross-scheme comparison. See Section 5.3.1.

Kronecker product. Composition that combines a scheme A at $\langle n_1, m_1, p_1 \rangle$ with a scheme B at $\langle n_2, m_2, p_2 \rangle$ into a scheme at $\langle n_1 n_2, m_1 m_2, p_1 p_2 \rangle$ whose rank is exactly $r_1 \cdot r_2$. Strassen recursion is K iterations of the Strassen $\langle 2, 2, 2 \rangle = 7$ atom Kronecker-composed with itself, giving $\langle 2^K, 2^K, 2^K \rangle$ at rank 7^K . See Section 5.2.

Permutation. Derivation that rewrites a scheme via an S_3 permutation of its three axes. The shapes $\langle n, m, p \rangle$, $\langle m, p, n \rangle$, and $\langle p, n, m \rangle$ (cyclic), together with their reflections, form a six-element orbit of equivalent problems. A permutation rewrite of (U, V, W) at $\langle n, m, p \rangle$ produces an equivalent scheme at the permuted shape with the same rank — so the catalog only stores one canonical representative per orbit (the sorted form $n \leq m \leq p$), and the search re-derives the other five on demand.

Axis-flip. Derivation that reverses the row order on one or more of the three axes (A, B, C). The eight resulting variants (3-bit mask, including the identity) of a scheme at $\langle n, m, p \rangle$ all have the same rank but generally distinct addition counts; they form an orbit under the $\mathbb{Z}_2^3$ sub-group of the natural symmetry group. The canonical form per orbit is chosen by lex-smallest (U, V, W) after the flip; non-canonical orbit members are not stored on disk but are emitted by the search via the `AxisFlip` lineage node. See Section 5.3.2.

Isotropy + coordinate change. The mathematically precise names for the symmetry actions on a bilinear-algorithm tensor. The *coordinate change* group at shape $\langle n, m, p \rangle$ is $GL_n(K) \times GL_m(K) \times GL_p(K)$ (basis change on each of the three index sets), combined with the discrete $S_3 \times \mathbb{Z}_2$ on top (axis permutation + the swap-and-transpose involution). The *isotropy* group of a given rank- r decomposition is the sub-group of the coordinate changes that fixes the decomposition — e.g. Burichenko 2014 shows the isotropy of Strassen $\langle 2, 2, 2 \rangle = 7$ has order 36, while de Groote 1978 shows ALL rank-7 $\langle 2, 2, 2 \rangle$ decompositions form a single 9-dimensional $GL_2(K)^3$ orbit.

Our *axis-flip* and *permutation* compositions (above) are the DISCRETE sub-actions of coordinate change — $\mathbb{Z}_2^3$ for axis flip, S_3 for axis permutation. The continuous $GL_n(K)^3$ action is not used in our search but is the right framework for discussing concrete representatives: different members of a single GL_n^3 -orbit can differ in addition count (Strassen 1969 has 18 raw additions, Strassen–Winograd 1971 has 15 scheduled — both in the same orbit), and finding new $\{-1, 0, +1\}$ -coefficient representatives may yield better-performing recombination bases. See Section 5.3.1.

Commutative vs non-commutative. Whether the underlying multiplication $x \cdot y$ is required to equal $y \cdot x$. Concretely:

- *Non-commutative* (NC) schemes (notation $K\langle n, m, p \rangle$) are valid for matrix multiplication over any ring — including the recursive case where the “scalars” are themselves matrices. The bilinear products $M_k = \langle U_k, A \rangle \cdot \langle V_k, B \rangle$ respect the left/right order. Strassen, Laderman, Smirnov, AlphaTensor are NC.
- *Commutative* (C, notation $K\langle n, m, p \rangle^c$) schemes use products of the form $M_k = \langle U_k, A \rangle \cdot \langle V_k, B \rangle$ where the two linear forms can be interchanged — they are valid only when the underlying scalars actually commute. Waksman 1970, Rosowski 2019 Thm 2/3 (= Islam Gen. Waksman), Makarov 1986 are commutative.

Critically: commutative schemes do *not* lift to recursive matmul over an NC ring. They are valid for scalar matmul and correspond to a strictly weaker model, with strictly fewer products in some shapes (e.g. $\langle 3, 3, 3 \rangle$ reaches 21 commutative vs 23 NC).

Shape $\langle n, m, p \rangle$. A matrix-product shape with $A \in K^{n \times m}, B \in K^{m \times p}$, giving $C \in K^{n \times p}$. We always tag the algebra (see Section 3): $K\langle n, m, p \rangle$ for NC, $K\langle n, m, p \rangle^c$ for commutative; ranks reported as $K\langle n, m, p \rangle:r$ or $K\langle n, m, p \rangle:m$ as needed. The bare $\langle n, m, p \rangle$ form is reserved for contexts where the algebra has just been named and we do not want to repeat the tag.

3 Notation and field discipline

A bilinear algorithm computing the product of an $n \times m$ matrix A by an $m \times p$ matrix B consists of three factor matrices $U \in K^{nm \times r}, V \in K^{mp \times r}, W \in K^{np \times r}$ over a field (or ring) K , such that for every A, B :

$$(AB)_{i,j} = \sum_{k=1}^r W_{(i,j),k} \left(\sum_{a,b} U_{(a,b),k} A_{a,b} \right) \left(\sum_{a',b'} V_{(a',b'),k} B_{a',b'} \right).$$

The integer r is the *rank* (a.k.a. *multiplication count*) of the algorithm. The *addition count* is the number of nonzero coefficients in (U, V, W) beyond the bare product multiplications, with conventions following Probert–Fischer 1976.

Shape notation. A bare $\langle n, m, p \rangle$ is not sufficient – the rank depends on the algebra. We adopt:

- $K\langle n, m, p \rangle$: non-commutative matmul over the field K . E.g. $\mathbb{R}\langle 7, 7, 7 \rangle = 250$.
- $K\langle n, m, p \rangle^c$ (superscript- c): commutative matmul over K . E.g. $\mathbb{R}\langle 3, 3, 3 \rangle^c = 21$ (Rosowski).

The value after $=$ is always the *rank* — the number of scalar multiplications, which is the quantity that compounds into an ω bound under recursion. Addition counts are tracked separately (Section 4); on disk the rank is the `_r{rank}` filename token (additions are recomputed from the factor matrices, not encoded in the name).

The fields we track.

$\mathbb{Z}, \mathbb{Q}, \mathbb{R}$ Characteristic-zero and non-commutative-friendly (they lift to recursive matmul). We keep them *distinct*, first-class fields — not a single cluster — and use the inclusion $\mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$ only to compute which fields a scheme satisfies. Ternary-integer schemes (ZT, coefficients in $\{-1, 0, 1\}$) are a first-class sub-class of $\mathbb{Z}$.

$\mathbb{C}$ A complex extension. Real-valid schemes work immediately; in addition, AlphaEvolve’s complex-coefficient schemes live here. Its $\mathbb{C}\langle 4, 4, 4 \rangle = 48$ was subsequently rationalised by Dumas–Pernet–Sedoglavic [4], so that particular rank no longer separates $\mathbb{C}$ from $\mathbb{Q}/\mathbb{R}$ — a reminder that field-separation examples are date-stamped claims.

$\mathbb{F}_2$ Characteristic-2. AlphaTensor’s $\mathbb{F}_2\langle 4, 4, 4 \rangle = 47$ lives here. Integer-valid schemes *do* reduce mod 2 to give $\mathbb{F}_2$ schemes, but our current widening sweep does not yet surface that exhaustively – see Section 11.

$\mathbb{F}_3$ Characteristic-3 (GF(3)) — “ternary modular”, not to be confused with the ternary-integer ZT sub-class of $\mathbb{Z}$. We track it so the discipline is genuinely field-agnostic, though coverage is currently thin: a dedicated $\mathbb{F}_3$ search is open (Section 11).

Commutative axis. Independent of the field choice, an algorithm may or may not assume that the entries of A and B commute. Commutative-only schemes (Waksman, Rosowski, Makarov 1986) do *not* lift to recursive matmul over non-commutative rings: they save bilinear products by exploiting the freedom $ab = ba$ that disappears as soon as A and B are themselves matrix blocks. They remain valid for scalar matmul. Rosowski 2019 makes the recursive side of this explicit: a commutative scheme *can* be applied recursively, but only while the base ring stays commutative (the linear combinations of products are themselves bilinear, which is what permits the recursion). His cubic bound is $K\langle n, n, n \rangle^c \leq n(n+1)(n+2)/2 \approx n^3/2$ (Theorem 5, n odd) — about half the n^3 scalar products, and below the best non-commutative rank at small sizes ($\mathbb{R}\langle 3, 3, 3 \rangle^c = 21$ versus Laderman’s $\mathbb{R}\langle 3, 3, 3 \rangle = 23$). The win is only this constant factor: because the recursion never leaves the commutative ring it does *not* lift to matrix-block recursion, so unlike Strassen’s $K\langle 2, 2, 2 \rangle = 7$ it does not lower ω over non-commutative rings. The catalog flags every commutative scheme with `"commutative": true` in the JSON; the comparison tables of Section 10 carry a separate commutative column for each field. Cross-contamination silently produces wrong “wins” against historical non-commutative tables; the canonical example is comparing $\mathbb{R}\langle 3, 3, 3 \rangle^c = 21$ (Rosowski, commutative) to $\mathbb{R}\langle 3, 3, 3 \rangle = 23$ (Laderman, non-commutative) as if they were the same metric.

We require every rank claim in the paper to carry both a field tag and a commutativity tag. The reader can recover the metric from the shape.

4 Catalog architecture

The catalog has two faces. At the top are a few *aggregated* JSON files under `docs/`: the manifest `docs/catalog.json` (one row per elected scheme, consumed by the web browser and the comparison tooling), alongside `docs/cited-bounds.json` and `docs/derived-bounds.json`. These are *generated* from the source of truth beneath them — a tree of per-scheme JSON files under `src/main/resources/schemes/sectionN/`, organised by the maximum of the three shape dimensions, each representing one algorithm at one shape, in one field, with one commutativity tag.

4.1 Three layers

Explicit schemes. JSON files that carry the full (U, V, W) factor matrices. Loaded by `SchemeIO.read` into a `NonCubicBilinearAlgorithm` record. Every explicit scheme passes `Verifier.passesRandom` at load time. The filename encodes the source, the shape, the rank, a content hash, and the field:

`{source}-{provenance}_nxm_xp_r{rank}_h{hash}[_field].json`

Example: `alphenator-Z_2x3x3_r15_h1b2c3d4.json`, `perminov-ZT_5x5x5_r93_h9f0a1e2.json`. The addition count is *not* in the name — it is recomputable from (U, V, W) ; the `_h{hash}` token is a short hash of the factor matrices giving each scheme a stable identity independent of how it was named upstream.

Cited bounds. A central JSON file `docs/cited-bounds.json` records rank claims whose explicit factor matrices we do not (yet) hold. Each entry has shape, field, commutativity, rank, addition count (or `null`), source citation, and discovery status. This lets us show the result in comparison tables without pretending to have the scheme. The cited layer is generated by `GenerateCitedBounds.java`.

Derived bounds. A separate JSON file `docs/derived-bounds.json` records rank claims that our own constructors can re-derive on demand from formulas (Waksman 1970, Rosowski 2019 Theorem 2/3, Pan 1980 trilinear aggregation, Hopcroft–Kerr 1971 $\langle 2, b, c \rangle$). Generated by

`GenerateDerivedBounds.java`; rebuilds whenever the underlying formula changes. This is the layer that grows as we “move” bounds from the cited tier to the derived tier by reproducing the original construction.

4.2 Lineage records

The *lineage* of a scheme is the exact sequence of composition steps that produced it, and it is **replayable**: a `LineageReplayer` reconstructs the full concrete (U, V, W) factor matrices from the lineage alone. This is essential — it lets the catalog store a large scheme as a lineage-only *stub* (no factor matrices on disk) and rebuild them on demand, and it makes every composed scheme reproducible from its building blocks rather than trusted as opaque numbers. The lineage is a sealed-node DAG with operations matching the composition primitives of Section 5: `Leaf`, `KronProduct`, `KronChain`, `ConcatRight`, `ConcatBelow`, `Recombination`, `RecombinationWithPair`, `Dce`, `Transpose`, `AxisFlip`, `AxisPermute`, and `DisjointSum` (the last reserved for imported disjoint-sum provenance and currently used by no scheme; see Section 5.8). The lineage is serialised twice: as a human-readable function-call form (e.g. `ConcatRight(KronProduct(strassen winograd), SAME0)`) and as a machine-readable JSON DAG with shared-subtree references.

The lineage is load-bearing for three reasons:

1. **Independent audit.** Because replay is independent of any stored matrices, re-deriving from leaves and comparing catches silent corruption of an on-disk (U, V, W) .
2. **Search-without-materialisation.** The frontier-closure search (Section 9) writes a strategy descriptor — effectively a lineage stub — before deciding whether to materialise. This lets the search iterate without paying for materialisation.
3. **Attribution archaeology.** When a bulk-imported catalog (FMM, Perminov) re-encodes a scheme originally due to Smirnov or Sedoglavic, the lineage records the importing source while a separate `attribution_for_rank` field records the earliest source that established the bound. Comparison tables pull from the latter.

4.3 Verification

Four verification modes run independently of the search loop.

Spot-check. `Verifier.passesRandomMatmulSpotCheck` samples random A, B , evaluates the scheme via its (U, V, W) , and compares against the direct product — $O(Sr(nm+mp+np))$ for S samples. Cheap enough to run at scheme load time, after every materialisation, and as a nightly catalog-wide sweep; probabilistic, but a few samples over a field make a false pass vanishingly unlikely.

Symbolic. `SymbolicVerifier` checks the trilinear identity *exactly* (an algebraic equality, no floating-point epsilon) and additionally rejects any coefficient outside the declared algebra. Conclusive, but $O(rnmmpnp)$ — the full tensor, $O(rn^6)$ in the cubic case — so it is reserved for the small-dimension schemes where that is affordable.

Lineage replay. `LineageReplayer` re-constructs the scheme from its lineage record, requiring the same (U, V, W) . Catches divergence between the on-disk scheme and the formula it claims to instantiate.

Field widening. `FieldWideningSweep` attempts to re-tag schemes claimed for a wide field with the narrower field they actually live in (e.g. a $\mathbb{R}$ -tagged scheme whose coefficients are all in $\mathbb{Q}$, or all in $\{0, \pm 1\}$ hence $\mathbb{Z}$).

5 Derivation strategies

A *derivation* strategy takes one or more catalog entries and produces a new bilinear algorithm at a target shape — by combining them upward (*composition*: Kronecker, concatenation), splitting a larger target onto a smaller scheme (*decomposition*, projection), or locally reducing the product set (peel, pair fusion). The strategies below cover what the catalog’s search and materialiser currently know how to do; each has a different cost structure and a different range of shapes where it wins.

We adopt the convention that the *outer* scheme is the one whose bilinear structure is reused, and the *sub-schemes* (or *leaves*) are the ones used to compute each of the outer scheme’s bilinear products at a smaller shape.

5.1 Axis concatenation (block-additive composition)

If two schemes share two of the three axes, they can be concatenated along the third: a $\langle n, m, p_1 \rangle$ scheme of rank r_1 and a $\langle n, m, p_2 \rangle$ scheme of rank r_2 compose into a $\langle n, m, p_1 + p_2 \rangle$ scheme of rank $r_1 + r_2$, simply by computing the left and right halves of the result independently. Analogous concatenations exist along the n and m axes. This is sometimes referred to as “plain split along a dimension”; it should not be confused with the τ -theorem disjoint sums of Section 5.8, which are additive over the tensor but not over a single axis.

The operator is trivial but pulls weight: at frontier-closure time it is the cheapest way to cover “rectangular” shapes ($\langle n, m, p \rangle$ with one large axis) from cubic catalog entries.

5.2 Kronecker product (multiplicative composition)

Given a scheme A at $\langle n_1, m_1, p_1 \rangle$ of rank r_1 and a scheme B at $\langle n_2, m_2, p_2 \rangle$ of rank r_2 , the Kronecker product $A \otimes B$ is a scheme at $\langle n_1 n_2, m_1 m_2, p_1 p_2 \rangle$ of rank $r_1 r_2$. This is the classical recursive substitution behind Strassen’s $\omega \leq \log_2 7$ bound: applying Strassen recursively to a power-of-two $\langle 2^k, 2^k, 2^k \rangle$ target gives 7^k bilinear products.

The catalog exposes Kronecker as a composition operator and the search enumerates 2-fold Kronecker chains over the pool. Higher-arity chains are derivable but not currently enumerated exhaustively — they appear via repeated 2-fold composition.

5.2.1 Serendipitous product (bud decomposition)

The serendipitous product is a multiplicative composition that, unlike the Kronecker product, can land *below* the naïve rank product $r_1 r_2$. It was introduced by Smith and generalised by Sedoglavic; we follow the bud-based formalisation of Perminov.¹ The same fusion was independently formalised by Kauers, Moosbauer and Wood [8] under the name “divide less, conquer more”: recursive calls that share an input, or whose output is used in several positions, are merged into a single larger matrix multiplication—our *U*- and *W*-buds, respectively—so that fewer recursion levels are needed and the effective exponent drops below what the tensor rank alone suggests (they report $\omega(\langle 6, 6, 6 \rangle) \leq 2.8019$). Notably, their route to *obtaining* bud-rich decompositions is a *search*: a flip-graph search maximising the number of shareable $\langle 1, 1, k \rangle$, $\langle 1, \ell, 1 \rangle$, $\langle m, 1, 1 \rangle$ sub-blocks under a non-overlap constraint—the same objective as our bud-richness selection—supplemented by *random* elements of de Groote’s symmetry group for support reduction; their $\langle 6, 6, 6 \rangle$ scheme is obtained by analysing a Moosbauer–Poole rank-153 decomposition and re-searching for one richer in buds, and they report no general *construction* of such structure. Whether a deterministic constructor (a targeted basis change in the spirit of the de Groote solve used for kin-row unification by Schwartz and Zwecher [19]) could replace this search is open.

¹A. I. Perminov, *Meta Flip Graph meets Serendipitous Product: new Fast Matrix Multiplication results*, draft, 2026. Definitions 2.9–2.12 and §2.6.

The key object is a *bud*: a pair (or group) of rank-one terms of a scheme that share the *same* u vector (or the same v , or the same w), where “same” is up to scaling, since a rank-one tensor $u \otimes v \otimes w$ is invariant under $u \mapsto \lambda u$, $w \mapsto \lambda^{-1} w$. Buds are basis-free: the shared vector may be any linear combination of input (or output) entries, not a single coordinate. Grouping the r terms of a scheme by their buds decomposes it into *elementary matmul tensors*

$$\mathcal{T} = \sum_i S_i \langle N_i, M_i, P_i \rangle, \quad r = \sum_i S_i N_i M_i P_i,$$

where a U -bud of k terms is an $\langle 1, 1, k \rangle$ block, a V -bud a $\langle k, 1, 1 \rangle$ block, a W -bud a $\langle 1, k, 1 \rangle$ block, combined buds give blocks such as $\langle 2, 1, 2 \rangle$, and every term in no bud is a trivial $\langle 1, 1, 1 \rangle$.

Given a second scheme $\langle n_2, m_2, p_2 \rangle$, the serendipitous product applies it to each elementary block independently:

$$\mathcal{T} \otimes \langle n_2, m_2, p_2 \rangle = \sum_i S_i \langle N_i n_2, M_i m_2, P_i p_2 \rangle,$$

and – crucially – each enlarged block is realised by the *best known* scheme for its format rather than by the naïve product, so

$$r_s = \sum_i S_i \cdot R(\langle N_i n_2, M_i m_2, P_i p_2 \rangle).$$

A saving over the Kronecker product arises whenever $R(\langle N_i n_2, M_i m_2, P_i p_2 \rangle) < N_i M_i P_i \cdot R(\langle n_2, m_2, p_2 \rangle)$ for at least one block; for trivial blocks the two coincide, so a scheme with no buds reduces exactly to the Kronecker product.

Worked example: $\langle 4, 8, 12 \rangle : 272$. The Hopcroft–Kerr $\langle 2, 3, 4 \rangle : 20$ scheme has four U -buds among its twenty terms, so it decomposes as $12 \cdot \langle 1, 1, 1 \rangle + 4 \cdot \langle 1, 1, 2 \rangle$ (note $12 + 4 \cdot 2 = 20$). Taking the serendipitous product with $\langle 2, 4, 2 \rangle$ and using $R(\langle 2, 4, 2 \rangle) = 14$ and $R(\langle 2, 4, 4 \rangle) = 26$,

$$\langle 2, 3, 4 \rangle \otimes \langle 2, 4, 2 \rangle = 12 \cdot \langle 2, 4, 2 \rangle + 4 \cdot \langle 2, 4, 4 \rangle, \quad r_s = 12 \cdot 14 + 4 \cdot 26 = 272,$$

a scheme for $\langle 4, 12, 8 \rangle$ (equivalently $\langle 4, 8, 12 \rangle$ up to axis permutation), below the Kronecker value $20 \cdot 14 = 280$. The shorthand “ $(\langle 2, 3, 4 \rangle : 20 - 8) \otimes \langle 2, 4, 2 \rangle + 4 \langle 2, 4, 4 \rangle$ ” used by the FMM catalog [20] records exactly this: eight of the twenty terms live in buds and are promoted to the four $\langle 2, 4, 4 \rangle$ blocks, the other twelve stay as $\langle 2, 4, 2 \rangle$.

Minimal example, multiplication by multiplication. The smallest serendipitous saving *derives Strassen*. Take the base $\langle 1, 1, 2 \rangle$ — a row times a 1×2 row, products $a b_1$ and $a b_2$; both share the single A -form a , so they form one U -bud of size 2. Multiplying by $\langle 2, 2, 1 \rangle$ gives target $\langle 2, 2, 2 \rangle$, and the bud fuses the two inner copies into one enlarged $\langle 2, 2, 2 \rangle$:

$$r_s = 1 \cdot R(\langle 2, 2, 2 \rangle) = 7 < 2 \cdot R(\langle 2, 2, 1 \rangle) = 8.$$

As a plain multiplication list, the Kronecker product runs the two $\langle 2, 2, 1 \rangle$ matrix–vector blocks independently — eight products, the naïve $\langle 2, 2, 2 \rangle$ — and both blocks reuse the *same* A entries; that reuse *is* the bud:

$$\begin{aligned} p1 &= a11 * b11 & p2 &= a12 * b21 & p3 &= a21 * b11 & p4 &= a22 * b21 & (c11 &= p1 + p2, & c21 &= p3 + p4) \\ p5 &= a11 * b12 & p6 &= a12 * b22 & p7 &= a21 * b12 & p8 &= a22 * b22 & (c12 &= p5 + p6, & c22 &= p7 + p8) \end{aligned}$$

Serendipity replaces them with Strassen’s seven, in which the shared A lets three products span *both* output columns at once:

$$\begin{aligned}
M1 &= (a11+a22)(b11+b22) & M2 &= (a21+a22)b11 & M3 &= a11(b12-b22) \\
M4 &= a22(b21-b11) & M5 &= (a11+a12)b22 & M6 &= (a21-a11)(b11+b12) \\
M7 &= (a12-a22)(b21+b22)
\end{aligned}$$

Exactly one product disappears ($8 \rightarrow 7$): the bud's shared A -form is what lets the cross-column combinations M_1, M_6, M_7 absorb the fourth, otherwise redundant pair. This is the whole mechanism in miniature — buds expose *which* products can be merged, and a sub-additive enlarged block ($\langle 2, 2, 2 \rangle : 7 < 8$) is what turns the merge into a saving.

Why bud-richness can be worth a higher rank. The rank-*minimal* scheme for a format is often the *wrong* base for a serendipitous product. A plain example with catalogued ranks builds $\langle 6, 6, 6 \rangle$ as $\langle 2, 2, 2 \rangle \otimes \langle 3, 3, 3 \rangle$, using $R(\langle 3, 3, 3 \rangle) = 23$ (Laderman) and $R(\langle 3, 3, 6 \rangle) = 40$ (Smirnov; note $40 < 2 \cdot 23$):

- The rank-minimal outer base, Strassen $\langle 2, 2, 2 \rangle : 7$, has *no* buds (its seven products use pairwise non-proportional u, v, w), so it decomposes as $7 \cdot \langle 1, 1, 1 \rangle$ and its serendipitous product is just the Kronecker product, $7 \cdot R(\langle 3, 3, 3 \rangle) = 7 \cdot 23 = 161$.
- The higher-rank *naïve* $\langle 2, 2, 2 \rangle : 8$ base is bud-rich: each of its four A -entries feeds two outputs, giving four U -buds, $4 \cdot \langle 1, 1, 2 \rangle$. Its serendipitous product fuses each bud with the inner $\langle 3, 3, 3 \rangle$ into a $\langle 3, 3, 6 \rangle$: $4 \cdot R(\langle 3, 3, 6 \rangle) = 4 \cdot 40 = 160$.

The bud-rich rank-8 base beats the rank-minimal rank-7 base ($160 < 161$) — but *only* through serendipity: its plain Kronecker product costs $8 \cdot 23 = 184$, the worst of the three. A catalog that keeps only the rank-minimal scheme per format discards exactly the base that wins here; recovering the 160 requires storing the bud-rich rank-8 variant alongside the rank-7 one.

How operations act on buds. Bud structure is not preserved uniformly by the composition operators, which is why it is tracked explicitly rather than recomputed from the rank-minimal scheme.

- *Rank reduction destroys buds.* A bud exists because several products share an input or output form; the standard route to lower rank replaces un-mixed products by independent linear combinations of entries, making the u, v, w vectors pairwise non-proportional. $\langle 2, 2, 2 \rangle : 7$ is the extreme case — zero buds, versus four for $\langle 2, 2, 2 \rangle : 8$; empirically every rank-minimal small-format scheme we hold is bud-free, and buds appear only on larger, less-mixed schemes.
- *The Kronecker product preserves and multiplies buds:* a term $u_i^1 \otimes u_j^2$ is proportional to $u_i^1 \otimes u_j^2$, iff both factors are proportional, so each bud of one factor lifts to a bud of the product. The serendipitous product is the bud-aware way of taking this Kronecker product.
- *Axis flip / permutation (the S_3 orbit) permutes the bud type:* a U -bud becomes a V - or W -bud under the corresponding relabelling, leaving the bud *count* unchanged — one orbit already exposes bud-richness on all three axes.
- *Projection / DCE can only shrink buds* (it deletes terms), while concatenating a scheme with a copy of itself *creates* U -buds (each product gains a twin with the same u but a fresh output column).

Which schemes to retain. Because r_s depends only on the bud *multiset* $\{S_i \langle N_i, M_i, P_i \rangle\}$, two schemes with the same multiset are interchangeable for every second factor: the store keys on the multiset (with rank and additions as tie-breakers), not on scheme identity. By the S_3 isotropy above, a “best- U , best- V , best- W ” triple is redundant — one orbit representative covers all three axes, with the orientation chosen at product time to align the bud axis with the contracted

axis of the second factor. What must be retained is the *Pareto frontier* in (rank, bud multiset): a higher-rank scheme is worth keeping iff its richer buds win some serendipitous product the lower-rank scheme cannot, while a scheme of no-better rank whose bud multiset is contained in another's is dominated and may be dropped. Independent schemes sharing the same maximal bud multiset collapse to the cheapest representative.

Aligning buds under isotropy. Because the Kronecker product multiplies proportionalities, the bud structure of $\mathcal{T}_1 \otimes \mathcal{T}_2$ depends on how the two factors' bud axes line up. A U -bud of size a in $\mathcal{T}_1$ aligned with a U -bud of size b in $\mathcal{T}_2$ (same axis) yields a U -bud of size ab in the product, since a shared product u requires both $u_i^1 \propto u_{i'}^1$ and $u_j^2 \propto u_{j'}^2$; a crossed pair (U against V) reinforces nothing and leaves only the size- a bud. The only isotropy that can realign buds is the discrete S_3 axis permutation: a basis change $u \mapsto Mu$ preserves proportionality ($Mu_i \propto Mu_j \Leftrightarrow u_i \propto u_j$), so the continuous GL part of the isotropy group leaves the bud partition *invariant* and only the axis relabelling moves buds between U , V and W . Two levers follow. For the (asymmetric) serendipitous product $\mathcal{T}_1 \otimes \langle n_2, m_2, p_2 \rangle$, where only $\mathcal{T}_1$'s buds are used, orienting $\mathcal{T}_1$ by S_3 selects which target axis carries the k -fold enlargement — a U -bud gives $\langle n_2, m_2, kp_2 \rangle$, a V -bud $\langle kn_2, m_2, p_2 \rangle$, whose R differ — and so changes r_s directly; the search should try all three orientations of the bud-provider. For a full $\mathcal{T}_1 \otimes \mathcal{T}_2$ reused as a base, aligning both factors' bud axes maximises the ab reinforcement and yields a bud-richer scheme for the next multiplication.

What it is, and is not. The serendipitous product is *exact* and *combinatorial*: it reads buds off the base scheme's factor matrices and looks up sub-block ranks; it requires neither numerical optimisation nor border rank. It differs from disjoint-sum / τ decomposition (§5.8), which splits the *target* tensor, and from pair fusion (§5.7), which fuses two equal-shape sub-products: here a single base scheme is multiplied by a second, and the saving comes from grouping the base's own terms into larger matmul blocks before multiplying. Our bud recognizer and bud-block constructor follow Perminov's §2.6 construction; the assembled scheme is certified by the exact symbolic verifier.

Smoothness governs the serendipitous hit rate. A serendipitous product needs the target to factor as $\langle n_1, m_1, p_1 \rangle \otimes \langle n_2, m_2, p_2 \rangle$ with *both* factors non-trivial, and the number of such factorisations of $\langle N, M, P \rangle$ is $d(N)d(M)d(P)$, the product of the per-axis divisor counts. A *smooth* format — one whose sides have many (distinct) prime factors, e.g. $30 = 2 \cdot 3 \cdot 5$ — therefore exposes far more base/inner splits than a prime or prime-power format, and each split is an independent chance for a bud-rich base to meet a sub-additive enlarged inner (both conditions must hold at once). One expects, and we observe, that serendipity wins disproportionately on smooth formats. Writing $\Omega(NMP)$ for the number of prime factors of NMP counted with multiplicity, Table 1 reports, over all 5456 non-commutative formats in the catalogue, how often the *best* scheme we hold is the serendipitous one.

Below $\Omega = 6$ no serendipitous scheme is ever best: the smallest non-trivial base/inner pair already needs $\Omega \geq 6$ (e.g. $\langle 2, 3, 7 \rangle \otimes \langle 3, 3, 3 \rangle$), so smaller formats admit no serendipitous decomposition at all. Above the floor the win rate tracks smoothness, peaking near $\Omega = 9$. The fall-off at $\Omega \geq 11$ is a coverage artefact — those formats are large, beyond our materialisation cap, where fewer serendipitous schemes are yet built — not a ceiling on the method. This concerns the practical per-format implied exponent, not the asymptotic ω ; operationally it gives the search a prioritisation rule: spend serendipitous effort on smooth formats first, and treat prime cubes as direct-construction targets where the method cannot help.

$\Omega(NMP)$	formats	serendipity best	%
≤ 5	2013	0	0.0
6	1122	6	0.5
7	979	12	1.2
8	654	11	1.7
9	389	9	2.3
10	178	3	1.7
≥ 11	121	0	0.0
all	5456	41	0.75

Table 1: How often the best known scheme for a format is a serendipitous product, bucketed by the smoothness $\Omega(NMP)$ of the format. No serendipitous scheme wins below $\Omega = 6$; above that floor the hit rate rises with smoothness and peaks near $\Omega = 9$. Formats where serendipity wins have mean $\Omega = 7.78$, against 6.29 for the whole population.

5.3 Decomposition: block-splitting and the sub-problem multiset

The operators above build *up*; the rest of this section builds *down*. *Decomposition* splits a large $\langle N, M, P \rangle$ product into a grid of blocks and computes that grid with a smaller *outer* scheme of shape $\langle n_o, m_o, p_o \rangle$ and rank r_o : cut each target axis into that many parts, view the target as an $n_o \times m_o$ by $m_o \times p_o$ block product, and let each of the outer scheme’s r_o bilinear products compute one block product. The basic expectation to hold onto: a $\langle 2, 2, 2 \rangle$ outer scheme turns the target into *seven* block products — seven sub-problems — and Laderman’s $\langle 3, 3, 3 \rangle$ into twenty-three.

The sub-problem multiset. List those sub-problems by shape, with multiplicity, and you get the *sub-problem multiset* of the decomposition. Since each is computed independently, the realised rank is just the sum of their ranks,

$$R(\langle N, M, P \rangle) \leq \sum_{\sigma \in \text{multiset}} R(\sigma),$$

so the multiset is the *only* thing that fixes the cost — order and labelling are irrelevant. When the parts are all equal the seven (or twenty-three) sub-problems are identical, the multiset is one shape repeated, and the sum collapses to $r_o \cdot R(\text{block})$ — exactly the Kronecker product (Section 5.2). The interesting regime is *unbalanced* parts, where the sub-problems differ. (This is a different multiset from the *bud* multiset of Section 5.2.1, which groups a scheme’s own rank-one terms; both are “multisets” but of different things, and a more specific name for one of them may be worth adopting.)

Padding and the max. The realised size of each sub-product is where padding enters, and it is the heart of the matter. When a product *sums* two blocks of unequal size on an output axis, the smaller is embedded into the larger with zeros, so it is realised at the *max* of the two parts on that axis (the contraction axis takes the *min*); the padding zeros are never multiplied. A product that sums across an axis is therefore charged that axis’s *large* part, while a product that uses a *single* block stays at the part it touches. Hence the expensive corner $\langle M_1, N_1, P_1 \rangle$ is paid by every all-sum product, and the cheap all-small corner $\langle M_2, N_2, P_2 \rangle$ is reachable only by a product that is single-block on *every* axis.

It is this max that decides Strassen versus Winograd. At a fixed orientation both induce the *same* multiset (Section 5.3.1: the $\langle 3, 3, 3 \rangle$ example gives 30 either way) and differ only in additions. But Strassen’s seven products are all two-block sums, so every one pays the max on every axis and the all-small corner never appears; Winograd has two *single*-block products, which the axis-flip

orbit (Section 5.3.2) can steer onto the small corner, removing one max. So the padding-max — not the rank, not the additions — is *why* Winograd beats Strassen on unbalanced splits. **Which template** and **which orientation** are the two knobs; recombination with allocation (Section 5.4) searches over them.

5.3.1 Strassen, Winograd, and the structural difference

The two canonical rank-7 algorithms for $\mathbb{R}\langle 2, 2, 2 \rangle$ are Strassen 1969 and Winograd’s 1971 reformulation. This section gives the *generic* effect of using either as a $\langle 2, 2, 2 \rangle$ template over a split $\langle M_1+M_2, N_1+N_2, P_1+P_2 \rangle$: the seven sub-products it induces, sized by zero-sparing. At this simple (fixed-orientation) level the two schemes induce the *same* multiset of sub-problems and differ only in additions (Strassen 1969: 18; Winograd 1971: 15). The *axis-flip orbit* of Section 5.3.2 then re-orientates the template – which *does* change the multiplications, and is where the two schemes diverge.

Heterogeneous split and zero-sparing. Project a target onto a $\langle 2, 2, 2 \rangle$ template by splitting each axis in two, $M = M_1 + M_2$, $N = N_1 + N_2$, $P = P_1 + P_2$, with the larger part first and the smaller part at the bottom/right (real data top-left, any padding at the bottom). The four A -blocks then have shapes $A_{11} : M_1 \times N_1$, $A_{12} : M_1 \times N_2$, $A_{21} : M_2 \times N_1$, $A_{22} : M_2 \times N_2$, and likewise for B and C . Each product is a (sum of A -blocks) $\cdot$ (sum of B -blocks); to add blocks of unequal shape the smaller is embedded into the larger envelope with zeros. *Those zeros are never multiplied*: a product is realised at the size of its genuine nonzero overlap – rows = the max real rows of the left sum, columns = the max real columns of the right sum, inner = the *min* of the two sides. A product that touches only the smaller part of an axis is thereby reduced to a lower rank.

The seven products, sized (canonical labelling). With block “11” the larger part on every axis, each product’s realised shape under zero-sparing (“blk” = single block, “sum” = two-block sum) is:

Strassen		operands	realised size
M_1	$(A_{11}+A_{22})(B_{11}+B_{22})$	sum·sum	$\langle M_1, N_1, P_1 \rangle$
M_2	$(A_{21}+A_{22}) B_{11}$	sum·blk	$\langle M_2, N_1, P_1 \rangle$
M_3	$A_{11}(B_{12}-B_{22})$	blk·sum	$\langle M_1, N_1, P_2 \rangle$
M_4	$A_{22}(B_{21}-B_{11})$	blk·sum	$\langle M_2, N_2, P_1 \rangle$
M_5	$(A_{11}+A_{12}) B_{22}$	sum·blk	$\langle M_1, N_2, P_2 \rangle$
M_6	$(A_{21}-A_{11})(B_{11}+B_{12})$	sum·sum	$\langle M_1, N_1, P_1 \rangle$
M_7	$(A_{12}-A_{22})(B_{21}+B_{22})$	sum·sum	$\langle M_1, N_2, P_1 \rangle$

Winograd		operands	realised size
P_1	$A_{11} B_{11}$	blk·blk	$\langle M_1, N_1, P_1 \rangle$
P_2	$A_{12} B_{21}$	blk·blk	$\langle M_1, N_2, P_1 \rangle$
P_3	$(A_{21}+A_{22})(B_{12}-B_{11})$	sum·sum	$\langle M_2, N_1, P_1 \rangle$
P_4	$S_2 T_2$	sum·sum	$\langle M_1, N_1, P_1 \rangle$
P_5	$S_3 T_3$	sum·sum	$\langle M_1, N_1, P_2 \rangle$
P_6	$S_4 B_{22}$	sum·blk	$\langle M_1, N_2, P_2 \rangle$
P_7	$A_{22} T_4$	blk·sum	$\langle M_2, N_2, P_1 \rangle$

with $S_2=A_{21}+A_{22}-A_{11}$, $S_3=A_{11}-A_{21}$, $S_4=A_{11}+A_{12}-A_{21}-A_{22}$, $T_2=B_{11}-B_{12}+B_{22}$, $T_3=B_{22}-B_{12}$, $T_4=B_{11}-B_{12}-B_{21}+B_{22}$.

The generic multiset. Collecting either “realised size” column gives the same multiset for both schemes:

$$2\langle M_1, N_1, P_1 \rangle + \langle M_2, N_1, P_1 \rangle + \langle M_1, N_1, P_2 \rangle + \langle M_1, N_2, P_1 \rangle + \langle M_1, N_2, P_2 \rangle + \langle M_2, N_2, P_1 \rangle.$$

The largest corner $\langle M_1, N_1, P_1 \rangle$ appears twice, the all-small corner $\langle M_2, N_2, P_2 \rangle$ not at all, and the split is *not* symmetric: the contraction axis N carries its smaller part in three of the seven products while M and P carry theirs in two each – so one minimises the small-part products by making N the largest axis. With equal parts (even dimensions) the multiset collapses to $7\langle M/2, N/2, P/2 \rangle$, the textbook recursion. At this fixed orientation, then, the choice of Strassen vs Winograd changes only the additive schedule, not the multiplications; re-orienting the template (Section 5.3.2) is what changes the multiplications.

Worked example: $\langle 3, 3, 3 \rangle$. Split $3 = 2+1$ on every axis ($M_1=N_1=P_1=2$, $M_2=N_2=P_2=1$). Both schemes yield

$$2\langle 2, 2, 2 \rangle + \langle 1, 2, 2 \rangle + \langle 2, 2, 1 \rangle + \langle 2, 1, 2 \rangle + \langle 2, 1, 1 \rangle + \langle 1, 1, 2 \rangle,$$

i.e. $2 \cdot 7 + 4 + 4 + 4 + 2 + 2 = 30$ leaf multiplications (recursing each $\langle 2, 2, 2 \rangle$ with the same rank-7 scheme) – against $7 \cdot 7 = 49$ for the naive pad-to-even $\langle 4, 4, 4 \rangle$ embedding and $3^3 = 27$ for the schoolbook product. The $49 \rightarrow 30$ gap is the zero-sparing; the further drop available by re-orienting the template is deferred to Section 5.3.2.

Why one needs a *set* of bases. Strassen and Winograd are only two points in the space of rank-7 $\langle 2, 2, 2 \rangle$ schemes, and — as the axis-flip orbit will show — what matters at an unbalanced split is the per-product block/sum structure (how many single-block products a scheme has, hence which sub-problem multisets its orbit can reach). To exploit this fully the pool should carry a *set* of rank-7 $\langle 2, 2, 2 \rangle$ bases that are inequivalent in their multiset behaviour, not just the two textbook ones; generating that set means enumerating the rank-7 normal forms (Heun 1994; de Groote’s classification) and keeping one representative per axis-flip orbit (Section 5.3.2). For $\langle 2, 2, 2 \rangle$ this enumeration is small and we do it exhaustively. The same idea applies to any base shape — a richer base set gives the recombination more multisets to choose from — but the space of rank- r schemes at larger shapes ($\langle 3, 3, 3 \rangle$ and up) is far too large to enumerate, so there we fall back on the imported/known schemes plus the axis-flip orbit of each.

5.3.2 Axis-flip orbit

For any bilinear scheme (U, V, W) of shape $\langle n, m, p \rangle$, reversing the row order of any of the three factor matrices gives another valid scheme of the same shape and the same rank. Composing the three independent axis-flips yields an orbit of size 8 per scheme (the elementary abelian 2^3).

Pure axis-flip is a trivial relabeling under uniform allocations (the orbit collapses) but becomes non-trivial under *unbalanced* allocations such as the $(9, 8) \times \dots$ split above. Different axis-flip orbit members give rise to different padding patterns and hence different total ranks. Section 5.4 explains why this matters at the recombination step: an axis-flip of the outer scheme corresponds to permuting which block-index in the target receives the padding penalty.

We canonicalise the pool of outer schemes by axis-flip orbit (picking one representative per orbit) and then re-enumerate the mask at search time, so the on-disk catalog stays compact while the search retains full coverage. The mask enumeration uses an analytical short-cut (per-product block-support bitmasks; see Section 9) so 8-mask coverage costs only a constant factor over single-mask evaluation.

Why the orbit separates Strassen from Winograd. This is exactly where the two rank-7 templates of Section 5.3.1 stop being interchangeable. At a fixed orientation they share the generic multiset above – two full $\langle M_1, N_1, P_1 \rangle$ products and no all-small $\langle M_2, N_2, P_2 \rangle$. Strassen’s seven products are all two-block sums, so under *any* orbit member each still spans a full part on every axis: Strassen keeps its two full products and can never realise the all-small corner. Winograd’s two single-block products ($A_{11}B_{11}$, $A_{12}B_{21}$) *can* be flipped onto the small blocks – one becomes $A_{22}B_{22} = \langle M_2, N_2, P_2 \rangle$ – so the best Winograd orbit member drops to a single full product and a strictly smaller multiset whenever the split is unbalanced (the orbit collapses, and the gain vanishes, only when the parts are equal). Concretely, $\langle 17, 17, 17 \rangle$ at the $(9, 8)^3$ allocation costs 2930 multiplications for the best Winograd orbit member – its lone all-small $\langle 8, 8, 8 \rangle$ product replacing one $\langle 9, 9, 9 \rangle$ – against 2940 for Strassen. It is a genuine *multiplication* saving, distinct from Winograd’s lower addition count.

5.4 Recombination with allocation

The central composition operator in the catalog. It uses a *small* catalog scheme — the outer *atom*, e.g. Strassen $\langle 2, 2, 2 \rangle$ — to multiply a *larger* target $\langle n, m, p \rangle$ by block-partitioning the target onto the atom’s grid. The atom’s dimensions $\langle n_o, m_o, p_o \rangle$ are small (typically 2 or 3, and $\leq n, m, p$) and fix only *how many* blocks each target axis is cut into, not their sizes. Concretely, given the atom of rank r_o , choose three *allocations*: a partition of the target’s n into n_o parts, of m into m_o parts, and of p into p_o parts. The atom then treats the target as an $n_o \times m_o$ by $m_o \times p_o$ block-matrix product: each of its r_o outer products becomes a smaller matmul on the corresponding blocks — a sub-shape fixed by the allocation — which is looked up in the catalog as a sub-scheme, and the assembled algorithm has rank $\sum_k r_{\text{sub-}k}$.

Uniform allocation reduces to Kronecker product (Section 5.2); non-uniform allocation gives strictly more flexibility – e.g. targeting $\langle 17, 17, 17 \rangle$ via Strassen with the allocation $(9, 8)^3$. Non-uniform recombination covers the gap left by Kronecker chains, at the price of *padding* – a sub-shape derived from two unequal blocks inherits the size of the larger block on the relevant axis.

5.5 Projection margin: a death-rate score

Projection is the downward operator: restrict a larger scheme to a smaller shape and dead-code-eliminate the products that die. A higher-rank parent can thereby project to a *lower*-rank child, and this has a clean closed form worth isolating, because it turns “keep the structurally better scheme, not the lower-rank one” into an explicit, computable criterion — the downward analogue of the bud score for the serendipitous product (Section 5.2.1).

A product survives the restriction iff *each* of its three factors U_k, V_k, W_k still has a nonzero inside the kept sub-block; it is dead-code-eliminated as soon as *any one* of them vanishes there. Fix an axis, say the first (n); only the two matrices carrying an n -index matter — U (input rows i) and W (output rows i). Write $U_n(k)$ and $W_n(k)$ for the sets of n -indices column k of U , resp. W , touches. Dropping index i kills product k exactly when *either* its α -form collapses ($U_n(k) = \{i\}$: all of U_k sits in the dropped input row, so $\alpha_k \equiv 0$ once that row is zeroed) *or* it writes only the dropped output ($W_n(k) = \{i\}$). The *private rank* at i is the count of such products,

$$\rho_n(i) = \# \{ k : U_n(k) = \{i\} \text{ or } W_n(k) = \{i\} \},$$

(an or, not an and: a product with $U_n(k) = \{i\}$, $W_n(k) = \{i'\}$ dies under dropping i *or* i'). Each axis uses two of the three matrices — $n \rightarrow \{U, W\}$, $m \rightarrow \{U, V\}$, $p \rightarrow \{V, W\}$ — and the *projection margin* of S (single index, best axis) is

$$\mu(S) = \max_{A \in \{n, m, p\}} \max_i \rho_A(i).$$

Projecting out the best single index then gives *exactly*

$$R_{\text{after}} = R_{\text{before}} - \mu(S),$$

and dropping a set D of δ indices on one axis generalises to $\mu_A^{(\delta)}(S) = \max_{|D|=\delta} \#\{k : \text{supp}_A(k) \subseteq D\}$, with $R_{\text{after}} = R_{\text{before}} - \mu^{(\delta)}$.

Projection therefore minimises $R - \mu$, *not* R : a scheme is worth keeping over a lower-rank rival whenever $R_S - \mu(S) < R_{S'} - \mu(S')$, i.e. on the (rank, projection margin) Pareto frontier. The canonical instance is $\langle 29, 30, 30 \rangle$: the rank-best parent, a flat $\langle 30, 30, 30 \rangle$:12688 (Schwartz–Zwecher 2025, an ALS-style decomposition), has $\mu = 62$ and projects to 12626; the higher-rank *structured* $\langle 30, 30, 30 \rangle$:12710 (Pan trilinear aggregation) has $\mu = 122$ and projects to 12588, matching FMM. Structure localises whole product-slabs to boundary slices (large ρ); a flat decomposition spreads every product across all indices (small ρ), so it projects worse despite the lower rank.

A small worked instance, and how to raise the margin. Ordering our small catalog by μ , the high-margin schemes are uniformly the *structured* ones (Pan trilinear aggregation) and the *flip-graph* ones (Perminov), never the flat rank-minimal decompositions. For example the Perminov $\langle 6, 6, 7 \rangle$:183 has $\mu = 26$ on the p -axis and projects to $\langle 6, 6, 6 \rangle$ at $183 - 26 = 157$ — acceptable, though not the catalog optimum 153; the rank-minimal $\langle 6, 6, 7 \rangle$ scheme projects worse. Two caveats sharpen the picture. First, at *small* dimension the flatter low-rank cube still wins projection outright, because the rank gap dwarfs the margin gap; the structured-cube advantage only *flips the winner* once the two ranks converge, as at $\langle 30, 30, 30 \rangle$ (12688 vs 12710, decided by $\mu = 62$ vs 122). Second — and parallel to the open question for buds — it is not obvious how to *construct* a scheme of the *same* rank but a higher μ . A promising route is the flip graph: a meta-flip (a rank-preserving move in the flip-graph of decompositions) leaves R fixed while reshuffling the product supports, and can concentrate more products onto a single boundary index — i.e. search the flip-graph for the μ -maximal representative of a rank class, exactly as one would search it for a bud-maximal one. We leave this generative direction to future work; here μ is used to *select* among existing schemes, not to synthesise new ones.

Downward propagation is acyclic and cheap. Unlike the serendipitous product, whose candidate bases form a multiplicative (divisor) lattice, projection’s dependency graph is a DAG ordered by strictly decreasing dimension: each scheme $\langle N, M, P \rangle$ projects only to its few δ -neighbours $\langle N - \delta, M, P \rangle, \dots$. The catalog therefore need not be re-swept to a fixed point round by round; a single newly-arrived scheme (an import, or a fresh search win) with $\mu > 0$ triggers a bounded *downward wave* — project it into its neighbours, and any neighbour that improves is itself enqueued for its own projections. Termination is immediate (dimension strictly decreases) and the work is event-driven: a new $\langle n, n, n \rangle$ cube refreshes exactly the rectangular and odd-cube shapes beneath it, nothing more. This makes projection the natural operator to fire on every catalog update (e.g. each daily Perminov / FMM sync), in contrast to the global recombination / serendipity passes.

5.6 Zero-peeling: input- and output-side (DIS09 γ_5)

When a recombination over-allocates an axis and pads the excess with zeros, the padded rows/columns are dead and the affected sub-products can be computed at a smaller shape. There are two directions, dual under the matmul tensor’s S_3 symmetry:

- *input-side*: a sub-product reads A only in some rows, or B only in some columns (the rest are padding zeros) — shrink its input extents;
- *output-side*: a sub-product’s W -support lands only in output positions that are peeled away after un-padding — shrink its output extents. This is the Drevet–Islam–Schost 2009 / Islam 2009 γ_5 reduction.

The peel-aware recombination shrinks each sub-product to the *elementwise minimum* of both directions, and the block-split search enumerates the (allocation, peel) patterns automatically (overshoot bounded per axis). The canonical γ_5 case – $\langle 3, 3, 3 \rangle$ via padded $\langle 4, 4, 4 \rangle$ Strassen, where the corner product collapses $\langle 2, 2, 2 \rangle = 7 \rightarrow \langle 1, 2, 1 \rangle = 2$ – is closed and tested.

Peeling is the second padding-fighter after axis-flip choice (Section 5.3.2); the two are complementary – flip distributes padding across products, peel removes it from a product whose slice was dead anyway – and, like axis-flip, it is *not* a sharing-discovery mechanism (Section 6).

Limit. Peeling alone does not close the odd-cubic gaps such as $\langle 17, 17, 17 \rangle = 2934$: there, no standard Strassen product has its support inside the peeled corner, so peeling leaves us at 2940. FMM-Lille’s 2934 recipe instead pair-fuses the diagonal $\langle 9, 9, 9 \rangle$ products and substitutes the corner with $\langle 8, 8, 8 \rangle$ – a pair-fusion (Section 5.7) plus outer-template substitution, not a peel.

5.7 Pair fusion (Pan trilinear aggregation)

Pan 1980 [13] observed that two same-shape sub-products arising at distinct positions in a larger scheme can be computed *jointly* via a trilinear aggregation that takes $abc + ab + bc + ca$ bilinear products in place of the naïve $2abc$. Cyclic pair-fusion identifies pairs of outer products with a compatible block-position pattern and substitutes a fused sub-scheme. This is the first derivation strategy that *does* introduce sharing across outer products – but only in a constrained, pattern-matching way, not as a generic discovery mechanism. The pair-fusion operator is implemented in `PairFusedRecombination`.

When it pays: a rank-density / ω threshold. Fusing is worthwhile only against the *best* independent computation, i.e. $abc + ab + bc + ca < 2R(\langle a, b, c \rangle)$ – not against the naïve $2abc$. Writing the leaf’s *rank density* $\rho = R(\langle a, b, c \rangle)/(abc)$, this is

$$\rho > \frac{1}{2} \left(1 + \frac{ab+bc+ca}{abc} \right) \approx \frac{1}{2}.$$

So pair-fusion helps exactly when the leaf is still “*fat*” – its rank has not been driven below about half the naïve abc . For a cubic leaf $\langle k, k, k \rangle = R$ with implied exponent $\omega_{\text{leaf}} = \log_k R$ (naïve $R = k^3 \Rightarrow \omega = 3$), the same condition reads

$$\omega_{\text{leaf}} > \omega^* := \log_k \frac{k^3+3k^2}{2} = 3 - \log_k 2 + \log_k \left(1 + \frac{3}{k} \right) \approx 3 - \log_k 2.$$

Worked examples. $\langle 7, 7, 7 \rangle = 249$ has $\rho = 249/343 = 0.726$ and $\omega_{\text{leaf}} = \log_7 249 = 2.835$, above its threshold $\omega^* = \log_7 245 = 2.827$: it *fuses* (pair cost $490 < 2 \cdot 249 = 498$, saving 8). By contrast $\langle 8, 9, 9 \rangle = 430$ has $\rho = 430/648 = 0.664$, just below its threshold 0.673: it is *not* fused (pair cost $873 > 2 \cdot 430 = 860$).

This is why pair-fusion is a *small-base, constant-factor* device: as $k \rightarrow \infty$ the threshold $\omega^* \rightarrow 3$ while good leaves have ω_{leaf} far below 3, so the saving vanishes asymptotically. (The *full* Pan trilinear aggregation – aggregating many products, not just pairs – is the genuinely ω -improving technique; what the catalog fuses is only the two-product instance.)

Why pairs, not triples. Given several same-shape leaves, why fuse them two at a time rather than three? Because there is *no* “fuse three *separate* products” primitive. The pair identity genuinely fuses two cyclically-related products $\langle a, b, c \rangle + \langle b, c, a \rangle$ at cost $abc + ab + bc + ca$. The $\frac{1}{3}$ of the full Pan/Islam aggregation does *not* come from fusing three products – it comes from one product’s own 3-fold cyclic index symmetry. A single $\langle n, n, n \rangle$ costs $\approx n^3/3 + O(n^2)$, and the $O(n^2)$ term is itself a *best-of* several Pan-family closed forms: $(n^3 + 12n^2 + 11n)/3$ (Islam 2009 [7] Prop. 1, republished as DIS09 §3) wins for moderate even n , but the smaller $\frac{15}{4}n^2$ quadratic of the $\frac{45}{4}n^2$ branch (Hadas–Schwartz 1982, sharpened by Schwartz–Zwecher 2025 [19]) overtakes it

for even $n \gtrsim 28$; odd n uses $(n^3 + 15n^2 + 14n - 6)/3$. The chooser takes the *minimum* over all of these (`PanTrilinearAggregation.bestPanTaBound`), so the large- n $\frac{45}{4}n^2$ regime is accounted for, not just the $12n^2$ branch. So the real per-leaf choice is shallow pair fusion ($\approx k^3/2$ per product, light $O(k^2)$ corrections) versus deep single-product TA ($\approx k^3/3$ per product, but $\sim 4\times$ the $O(k^2)$ corrections).

At a small leaf the corrections dominate and pairs win decisively. For the seven $\langle 7, 7, 7 \rangle$ leaves of a $\langle 14, 14, 14 \rangle$ (or padded $\langle 17, 17, 17 \rangle$) recombination:

option	per product	total (7 leaves)
solo (catalog SOTA)	249	1743
pair-fused	245	1719
full single-product TA	390	2730

Pair fusion (1719) beats both solo (1743) and full TA (2730). The deep TA’s $\frac{1}{3}$ only overcomes its heavier corrections at *even* $k \geq 18$; odd k carries the heavier $15k^2$ term and never wins in the practical range. This is why the catalog pair-fuses small leaves and reserves full TA for large even ones. The chooser `PairFusedRecombination.chooseBest` weighs all three options (`SOLO` / `PAIR_FUSED` / `FULL_TA`) per leaf and returns the cheapest.

5.8 Disjoint-sum decomposition (τ -theorem)

A target tensor can decompose as a sum of smaller matmul sub-tensors *at the tensor level* – not at a single-axis level – when the target admits a τ -theorem-style decomposition. The construction is constructive: each leg of the disjoint sum computes a smaller matmul, and the legs combine via the additive structure of the tensor. Schwartz–Zwecher 2025 [19] give explicit constructive disjoint-sum decompositions for *large* formats (e.g. $n = 44, 56, 60$); the FMM catalog [20] carries disjoint-sum schemes at smaller shapes such as $\langle 17, 17, 17 \rangle$ at rank 2934. (Our own $\langle 17, 17, 17 \rangle = 2930$ is *not* one of these — it is a Winograd recombination, Section 5.3.2, not a disjoint sum.)

We stress that *we do not currently leverage the τ -theorem anywhere*. The disjoint-sum schemes we hold (Schwartz–Zwecher and those FMM/Perminov build this way) are *imported as explicit factor matrices*, not reconstructed from a τ decomposition: no catalog entry carries a τ lineage, and every gap we initially attributed to a τ mechanism turned out, on inspection, to close (or not) by other means – Kronecker, recombination, or a fortunate cousin scheme. A `DisjointSum` lineage node (with optional `taLegs` groups) is *reserved* in the schema for this provenance but is exercised by no scheme today, and we have no search operator that constructs disjoint sums. Section 11 flags this as the largest open extension to the search.

5.9 Summary table of operators

The “no-sharing” rows are the ones that fall under the non-overlap property of Section 6. The “yes-sharing” rows are deliberately written as separate constructors so that the non-overlap invariant holds for every operator the generic recombination path can produce.

6 The non-overlap property and a methodological boundary

[TO BE REWORDED]

The composition operators of Sections 5.3.2 through 5.6 are subject to the following structural invariant.

Theorem 1 (Non-overlap). *Let O be an outer bilinear scheme of shape $\langle n_o, m_o, p_o \rangle$ with r_o products, and let $L_1, \dots, L_{r_o}$ be sub-schemes assigned via `Recombination.constructWithAllocation` to a target shape $\langle n, m, p \rangle$ with allocations $(\mathbf{a}_A, \mathbf{a}_B, \mathbf{a}_C)$ and an optional output peel pattern. The*

Table 2: Derivation strategies used by the catalog. “Sharing” is the operator’s ability to produce a scheme with fewer than $\sum_k r_{\text{sub-}k}$ bilinear products – see Section 6.

Operator	Source	Sharing?
Axis-flip orbit	folklore + this paper §5.3.2	no
Kronecker product	Strassen 1969	no
Axis concatenation	folklore	no
Recombination with allocation	Strassen-style block split	no
Output peel	DIS09 / Islam 2009 γ_5	no
Pair fusion (Pan TA)	Pan 1980	limited (paired only)
Disjoint-sum / $\tau^\dagger$	Pan 1980 / Schönhage 1981 / SZ 2025	n/a (imported)

[†] Not a search operator: disjoint-sum schemes are imported as explicit factor matrices; we do not construct them (Section 11).

resulting algorithm has rank exactly $\sum_k r_{L_k}$, and no two of its bilinear products are equal as bilinear forms.

Sketch. The construction is block-substitution: each outer product $k \in [1, r_o]$ is instantiated as the sub-scheme L_k wired into block-positions of (A, B, C) determined by the allocations and the outer scheme’s bilinear form. Two columns of the resulting (U, V, W) can be equal as bilinear forms only if (a) they come from the same outer product index AND (b) they correspond to the same column of L_k – which would require the same column of the sub-scheme to be wired to two different positions, contradicting the definition of block-substitution. The peel reduction (Section 5.6) removes dead rows of W but never identifies two columns. The axis-flip orbit (Section 5.3.2) is an outer transformation that does not change the rank or the column-distinctness. $\square$

Scope: the operations that *reduce* below the sum. The theorem is about pure block-substitution, so it governs Kronecker, concatenation, recombination-with-allocation, the axis-flip orbit and the peel. Three operations deliberately fall *outside* it by producing fewer than $\sum_k r_{L_k}$ products — and none is a counterexample, because none *discovers* sharing; each applies a known, *predict-time* reduction.

- **Pair fusion (Pan TA)** is not a composition operator at all: it is an optimisation on a decomposition’s product set, fusing complementary pairs through Pan’s trilinear-aggregation *identity*. The saving is a property of that fixed identity, counted before materialisation — TA does not break the invariant, it just is not a $\sum_k r_{L_k}$ construction.
- **The serendipitous product** replaces an enlarged sub-block by its catalogued best rank — again a lookup performed at predict time.
- **Downward projection** restricts a materialised parent and runs DCE; the resulting rank $R - \mu$ is exactly what the projection-margin score (Section 9) *predicts* from the parent’s support. The DCE is the same deterministic step in both the prediction and the materialisation, so nothing is discovered.

So the deeper invariant below — *predicted rank equals materialised rank, with no sharing found during materialisation* — holds for these three as well, even though the literal $\sum_k r_{L_k}$ equality of the theorem does not.

Theorem 1 has two practical consequences worth calling out.

Predicted rank equals materialised rank. The frontier-closure search (Section 9) propagates rank predictions $\hat{r} = \sum_k r_{\text{sub-}k}$ computed by summing sub-shape lookups in the catalog. The

materialised scheme has exactly that rank. There is no hidden discovery during materialisation; the materialisation step is faithful execution of whatever strategy the search picked. This is what lets us split the closure loop into a fast rank-only search phase and a lazy materialise-only phase without losing accuracy – see Section 9 for the algorithm.

A methodological boundary against hand-crafted schemes. Almost every hand-crafted small-format result in the literature exists *because* its author found a non-obvious shared bilinear core: Strassen’s $\mathbb{R}\langle 2, 2, 2 \rangle = 7$ beats 8 because two of his seven products do double duty across output cells; Laderman’s $\mathbb{R}\langle 3, 3, 3 \rangle = 23$ is the same story at larger scale; Smirnov, Pan, AlphaTensor, AlphaEvolve all hunt for non-obvious cross-product sharing. Theorem 1 says our *composition* operators do *not* hunt for such sharing – the wins come from picking the right composition of cataloged building blocks. (The reducing operators of Section 5.3 ff. — serendipity, pair fusion, projection — do exploit known reductions, but never discover new ones; see the scope remark above.)

This makes a “composition matches hand-crafted rank” outcome a *co-existence* claim, not a re-discovery claim. When the search reaches $\mathbb{Q}\langle 17, 17, 17 \rangle = 2930$ via a Winograd recombination at $(9, 8)^3$, it does so by a route structurally distinct from the hand-crafted disjoint-sum construction for the same shape (FMM-Lille, rank 2934): our scheme is a plain sum of independent sub-products and reproduces none of that construction’s shared bilinear identities. The catalog records both with distinct lineages and distinct `attribution_for_rank`.

The two approaches are complementary, not redundant. Composition expands the SOTA frontier without finding new bilinear identities; hand-crafted constructions find new bilinear identities that can be ingested as new outer schemes, immediately expanding what composition can do.

The invariant is current-state, not eternal. Theorem 1 depends on `constructWithAllocation` being pure block-substitution. Several extensions would break it: post-construction column fusion that identifies duplicate bilinear forms; random sign salting of sub-blocks that enriches the symmetry orbit; coefficient mixing between outer products that share a sub-shape. Any of these would let materialised rank be *lower* than predicted rank, which would be a feature – but would also break the search-rank-equals-materialised-rank invariant that the two-phase search of Section 9 relies on. The implementor of any such “salt” must either restrict it to the final materialisation step (so the search keeps its pure prediction) or re-derive predictions to account for the salt. The catalog documents this in `docs/notes/materialisation-and-overlap.md`.

We have considered whether to add such salt and decided against it for now: a pure search-time prediction is easier to reason about, easier to cache, and avoids a class of subtle correctness bugs at the search/materialise boundary. Hand-crafted-style sharing discoveries are deferred to the one dedicated constructor we have — pair fusion (Section 5.7) — rather than emerging from generic recombination; the disjoint-sum / τ route (Section 5.8) remains future work.

7 Balance versus imbalance: the records are unbalanced

A recurring heuristic — explicit in DIS09 and folklore well before — is that recombination should prefer *balanced* block splits and avoid unbalanced ones. We test this directly against the catalog and reach a sharper, two-sided conclusion: **balance is better on average, but the records are unbalanced**. A search that prunes unbalanced shapes is therefore average-case sound yet *record-blind* — it discards exactly the shapes that set the lowest exponents.

Throughout this section the per-shape exponent is $\omega(n, m, p) = 3 \ln R(n, m, p) / \ln(nmp)$, where R is the best catalogued non-commutative rank over $\mathbb{R}$ (catalog snapshot 2026-06).

7.1 On average, ω rises with imbalance

Table 3 buckets every catalogued shape $\langle n, m, p \rangle$ with $2 \leq n \leq m \leq p \leq 16$ by its imbalance $p - n$, and reports the median ω per bucket.

Table 3: Median ω over all catalogued shapes $2 \leq n \leq m \leq p \leq 16$, bucketed by imbalance $p - n$. The median climbs monotonically — a *typical* unbalanced shape is less efficient.

imbalance $p - n$	#shapes	median ω
0	15	2.8155
1	28	2.8148
2	39	2.8177
3	48	2.8180
4	55	2.8169
5	60	2.8193
6	63	2.8214
7	64	2.8229
8	63	2.8274
9	60	2.8299
10	55	2.8332
11	48	2.8402
12	39	2.8536
13	28	2.8616
14	15	2.8695

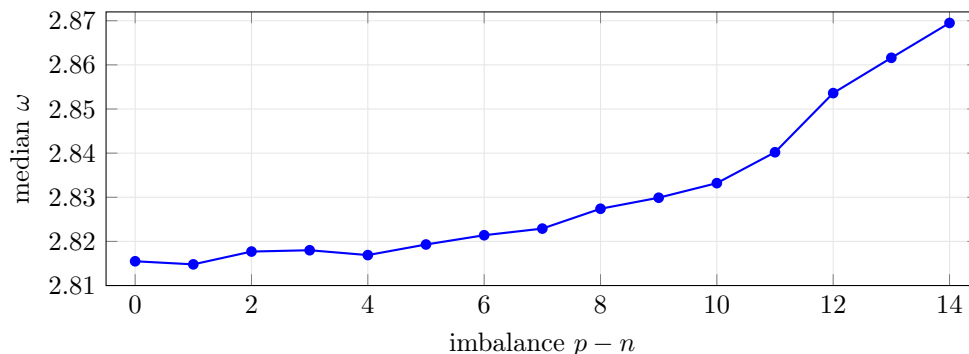


Figure 1: The data of Table 3 as a curve: median ω is flat (≈ 2.817) out to imbalance ~ 7 , then climbs steeply — the penalty for imbalance is convex, not linear.

The median rises from 2.8155 (balanced) to 2.8695 at imbalance 14. This is the average-case fact the avoid-unbalanced heuristic rests on.

7.2 ... but the lowest exponents are unbalanced

The aggregate hides the tail. Among the same shapes, the fifteen smallest ω are almost all unbalanced, and the global minimiser in this range is $\langle 3, 3, 6 \rangle$ at $\omega = 2.7743$ — below *every* cubic: $\langle 4, 4, 4 \rangle$ (2.7925), $\langle 8, 8, 8 \rangle$ (2.7974), $\langle 12, 12, 12 \rangle$ (2.7957). The low- ω frontier is dominated by the 2:1-ratio family $\langle 3, 3, 6 \rangle$, $\langle 6, 6, 12 \rangle$, $\langle 9, 9, 12 \rangle$, $\langle 5, 5, 12 \rangle$, $\langle 3, 4, 6 \rangle$. Pruning unbalanced splits removes precisely these record-holders.

7.3 Why the advantage is not extractable by outer splitting

It is tempting to conclude that a recombination should *seek* unbalanced splits so its sub-blocks land on these efficient shapes. The catalog says otherwise, and the mechanism is instructive. Consider $\langle 9, 9, 9 \rangle$ recombined by Strassen $\langle 2, 2, 2 \rangle = 7$: since $9 = 5 + 4 = 6 + 3$, the balanced split $(5, 4)^3$ and the unbalanced $(6, 3)^3$ both tile exactly (no padding). Table 4 puts their seven-product multisets side by side.

Table 4: Strassen $\langle 2, 2, 2 \rangle$ recombination of $\langle 9, 9, 9 \rangle$: the seven sub-products grouped by corner tier (the $2 \times 2 \times 2$ split realises one all-max corner, three two-max corners, and three one-max corners). Reading the Δ column top to bottom is the whole story: the unbalanced $(6, 3)^3$ split *saves* 63 on the three one-max corners (the $\langle 3, 3, 6 \rangle$ gems, 40 vs 61) but is mechanically forced to *pay* +60 on the single all-max corner ($\langle 6, 6, 6 \rangle = 153$ instead of $\langle 5, 5, 5 \rangle = 93$) plus +12 on the middle tier — net +9, so balance wins here.

corner tier (multiplicity)	$(5, 4)^3 \rightarrow 504$ shape	subtotal	$(6, 3)^3 \rightarrow 513$ shape	subtotal	Δ
all-max $\times 1$	$\langle 5, 5, 5 \rangle = 93$	93	$\langle 6, 6, 6 \rangle = 153$	153	+60
two-max $\times 3$	$\langle 5, 5, 4 \rangle = 76$	228	$\langle 6, 6, 3 \rangle = 80$	240	+12
one-max $\times 3$	$\langle 5, 4, 4 \rangle = 61$	183	$\langle 3, 3, 6 \rangle = 40$	120	-63
total		504		513	+9

A $\langle 2, 2, 2 \rangle$ split realises seven of the eight corners of $\{a_1, a_2\} \times \{b_1, b_2\} \times \{c_1, c_2\}$, and *one of them is the all-max corner* (max, max, max). The unbalanced split buys three cheap $\langle 3, 3, 6 \rangle$ -type corners (saving $3 \times (61 - 40) = 63$) but mechanically forces the all-max corner from $\langle 5, 5, 5 \rangle = 93$ up to $\langle 6, 6, 6 \rangle = 153$ (costing +60), plus +12 on the middle corners — net +9. Because R grows like size $^\omega$ with $\omega \approx 2.8$, pushing the max block from 5 to 6 costs more than the cheap corners recover. The efficiency of $\langle 3, 3, 6 \rangle$ is real, but a single block split cannot isolate it from its complementary all-max corner.

The corollary is constructive: an efficient unbalanced atom is exploited not by *splitting* a target down onto it, but by *composing up* from it via the Kronecker product, where the favourable ratio compounds instead of dragging a complementary max-corner along. (A Schönhage τ -disjoint sum would compound the ratio in the same way, but we do not use that route — see Section 5.8.)

7.4 Imbalance can win the split, too

Section 7.3 showed balance winning a single example; it is not the rule. We swept every target $\langle n, m, p \rangle$, $3 \leq n \leq m \leq p \leq 16$, computing for a fixed base the *exact* rank-minimising allocation by branch-and-bound (`AllocationOptimizer`: balanced incumbent + the admissible lower bound of Section 7.3, verified against exhaustive search), and flagged every target whose optimal split is strictly cheaper than the balanced split. The search is deliberately *single-base* and *non-recursive*: sub-blocks are costed by the catalog’s best rank as a black box, so a “win” means the unbalanced split beats its own balanced sibling for that base — not that it beats the global record.

Unbalanced splits win for 15 targets under Strassen $\langle 2, 2, 2 \rangle$ and 120 under Strassen–Winograd (whose asymmetric product supports reward imbalance far more often). The effect is not confined to rectangular targets: the cubic $\langle 13, 13, 13 \rangle$ is itself a win. Table 5 lists representatives.

The last row is the most imbalanced win we found in the sweep ($3 \leq n \leq m \leq p \leq 16$): for $\langle 5, 5, 12 \rangle$ under base $\langle 2, 2, 3 \rangle$ the optimal split of the size-12 axis is $(6, 5, 1)$ — it *peels a width-1 strip*. The products that touch that unit block compute $\langle \cdot, \cdot, 1 \rangle$ sub-shapes, whose rank is the naive product (here $R = 4, 6, 6, 9$), while the remaining products route through the cheap $\langle 3, 3, 6 \rangle = 40$ corner. The thin strip costs almost nothing yet lets the rest land on the efficient

Table 5: Targets where the rank-optimal single-base split is *unbalanced* (strictly cheaper than the balanced split). Single base, no recursion; sub-blocks costed by catalog SOTA. Savings are small but the existence is the point: a hard “always split balanced” rule is provably suboptimal.

base	target	balanced	optimal split	best
Strassen	$\langle 13, 13, 13 \rangle$	1434	$(5, 8)(6, 7)(6, 7)$	1432
Strassen	$\langle 6, 6, 10 \rangle$	252	$(3, 3)(3, 3)(4, 6)$	247
Strassen	$\langle 12, 12, 14 \rangle$	1281	$(6, 6)(6, 6)(6, 8)$	1271
Winograd	$\langle 9, 13, 15 \rangle$	1156	$(5, 4)(7, 6)(7, 8)$	1145
Winograd	$\langle 9, 14, 15 \rangle$	1237	$(4, 5)(7, 7)(8, 7)$	1226
$\langle 2, 2, 3 \rangle$	$\langle 5, 5, 12 \rangle$	235	$(2, 3)(2, 3)(6, 5, 1)$	232

$\langle 3, 3, 6 \rangle$ family — the unbalanced-extraction mechanism in its purest form, and precisely the allocation a balanced-only rule forbids.

So the practical rule is not “prefer balance” but “*search* the split, balance-first”: seed the branch-and-bound with the balanced allocation (an excellent, usually-optimal incumbent) and let an admissible lower bound prune the rest. When a known upper bound for the target exists — typically the Kronecker rank of a factorisation $\langle n, m, p \rangle = \langle n_1, m_1, p_1 \rangle \otimes \langle n_2, m_2, p_2 \rangle$ — it is used as the incumbent: if the base’s root lower bound already meets or exceeds it, the base cannot improve and the allocation sweep is dropped outright.

7.5 Open questions

Two extrapolations are untested here. (i) *Scale*: the catalog thins above max-dim 16, so Table 3 cannot yet say whether the average ω -vs-imbalance slope persists, flattens, or reverses for large N ; the asymptotic τ -theorem constructions suggest the unbalanced tail should only get richer. (ii) *Rectangular targets*: the metric above folds rectangular shapes into a single imbalance scalar $p - n$; a genuinely rectangular sweep (independent control of $m - n$ and $p - m$) may expose direction-dependent structure the cubic-centred view misses.

8 Recombination multisets of a base

Recombining a base $\langle n, m, p \rangle$ at a block decomposition $(n_1 + \dots + n_n, m_1 + \dots + m_m, p_1 + \dots + p_p)$ sends each of the base’s r products to a smaller matrix multiplication; the per-axis sub-dimension of a product is the size of the *largest* block it touches (capped by the min of the two relevant factor views). The resulting *multiset* of r sub-shapes is, for plain additive recombination, a *complete rank invariant*: any two routes that land on the same multiset have identical recombined rank $\sum_k r_{\text{sub-}k}$ (Section 9). Two facts make the multiset the right object to enumerate. First, it depends *only on the support pattern* (the zero/non-zero structure) of (U, V, W) , not on the coefficient values. Second, with each axis’s blocks ordered descending ($n_1 \geq n_2 \geq \dots$), each sub-dimension is one of the block sizes, so the multiset is *symbolic* in (n_i, m_j, p_k) — a product is tagged, per axis, by the block index it lands on. The concrete split $(9, 8)^3 \rightarrow \langle 17, 17, 17 \rangle$ used below is just one instantiation.

8.1 The orbit and a decoupling lemma

By de Groote [2] the rank-7 decompositions of $\langle 2, 2, 2 \rangle$ form a *single* $\text{GL}_2(\mathbb{Q})^3$ orbit (its discrete isotropy is analysed in [1]). Every scheme is therefore a change of basis of Strassen’s, acting on the factors by $U'_k = X^\top U_k Y^\top$, $V'_k = Y^{-\top} V_k Z^\top$, $W'_k = X^{-1} W_k Z^{-1}$ for $(X, Y, Z) \in \text{GL}_2 \times \text{GL}_2 \times \text{GL}_2$. A direct brute-force sweep of ternary schemes is hopeless ($\approx 39\text{k}$ candidate rank-one terms per product, 39k^7 paths). The structure that rescues it:

Lemma 2 (Per-axis decoupling). *The recombination sub-dimension on a given axis depends on only one of X, Y, Z , and only through that matrix’s column/row directions. Concretely, on the first axis,*

$$\text{sub}A_k = \text{big} \iff d_0^\top U_k \neq 0 \wedge \text{perp}(d_1)^\top W_k \neq 0, \quad X = [d_0 \mid d_1],$$

and analogously for the second and third axes via Y and Z — with the caveat that, by the transpose placement of the action, Y and Z act through their row directions (and the dual tests use adjugate columns), so the displayed column formula must not be transplanted verbatim.

The lemma holds because an invertible factor on the *other* two axes kills no rows or columns of a factor matrix, so the per-axis support is unaffected by it. Consequently the three axes are independent and the set of realisable multisets is the product of three per-axis pattern sets, zipped by product index.

8.2 Exact enumeration

Each per-axis pattern is piecewise-constant in the direction(s) and changes only when a direction meets a null-space of one of the fixed (integer) base factors. The critical directions are therefore finite and rational, so a finite integer-direction sweep enumerates *every* realisable pattern — exactly, with integer arithmetic (adjugates in place of inverses, so zero/non-zero tests are exact). For a two-part axis this is provably complete: the factor null-spaces are the only critical directions, and one generic direction covers the open stratum. For ≥ 3 -part axes completeness is certified only *empirically*, by direction-bound stability — a bound, not a proof. We emphasise that this is a *derivation*, not a search over schemes.

Result for $\langle 2, 2, 2 \rangle$. Over the full $\text{GL}_2(\mathbb{Q})^3$ orbit there are exactly **40** distinct canonical recombination multisets. The same set covers *every* rank-7 $\langle 2, 2, 2 \rangle$ scheme over any characteristic-0 field — not because the $\mathbb{Q}$ -orbit contains them (a scheme with irrational coefficients lies in the $\text{GL}_2(\mathbb{R})^3$ orbit only), but because the per-axis strata are cut by *rational* hyperplanes (the null-spaces of the integer Strassen factors), so every stratum that is nonempty over $\mathbb{R}$ or $\mathbb{C}$ already contains a rational point (canonical = quotiented by the base’s S_3 axis-symmetry; stable across direction bounds, and identical whether seeded from Strassen or Winograd, an independent check of de Groote’s single-orbit theorem). Of these, **6** are realised by ternary $\{-1, 0, 1\}$ schemes *within the ternary change-of-basis scope*: the count is stable under widening the change-of-basis alphabet and under the choice of seed, but is not certified exhaustive over all ternary schemes (one reachable only through a non-ternary rational change of basis could exist), so per our optimality discipline it is a lower bound, “at least 6”. The six are those of Strassen, Winograd, AlphaTensor- $\mathbb{Z}$, and Perminov, plus two we register as bases so the search can use them — one matching AlphaTensor- $\mathbb{F}_2$ ’s support, and a novel base whose multiset is $3 \cdot \langle n_1, m_1, p_1 \rangle + \langle n_1, m_2, p_2 \rangle + \langle n_2, m_1, p_2 \rangle + \langle n_2, m_2, p_1 \rangle + \langle n_2, m_2, p_2 \rangle$. The novel base is of independent interest: its triple cubic group $3 \cdot \langle n_1, m_1, p_1 \rangle$ admits 3-way trilinear aggregation [12], a cost lever Strassen’s own multiset (seven distinct sub-shapes) does not expose.

8.3 Generality

The construction is base-agnostic: `RecombinationMultisetOrbit` enumerates the multisets of any base $\langle n, m, p \rangle$ over $\text{GL}_n \times \text{GL}_m \times \text{GL}_p$, with the canonicalising group taken to be the base’s shape automorphisms (the full S_3 for a cube, the single $m \leftrightarrow p$ swap for $\langle 2, 3, 3 \rangle$, the trivial group for an all-distinct shape). It thus answers, for any candidate outer base, *which* sub-shape multisets — and hence which trilinear-aggregation and padding-avoidance opportunities — that base can ever expose to the frontier search.

9 Frontier-closure search

The frontier-closure search converges the catalog to a fixed point under the derivation operators of Section 5. It propagates rank improvements along a dependency DAG of shapes (below) rather than re-deriving everything from scratch; the loop terminates when no shape improves.

9.1 Two-phase structure

A naive closure interleaves *search* (find the best recombination) and *materialise* (build and write the concrete (U, V, W)) at every shape, every round. We split them:

Phase 1 – Search. Iterate rounds. For each shape, run the derivation search to find the best predicted rank $\hat{r} = \sum_k r_{\text{sub-}k}$. Strict improvements over the shape’s current best (catalog or overlay) are recorded in an in-memory *pending overlay*, keyed by shape. The overlay is consulted by the rank resolver in subsequent rounds, so a win at round N propagates to round $N + 1$ without writing anything to disk. Repeat until a round produces no new overlay entries.

Phase 2 – Materialise. Iterate the overlay in dependency order (smaller maximum-dimension first). For each entry, materialise the strategy via `Recombination.constructWithAllocation` or its concat / Kronecker analogue. Sub-product lookups are served from a composed `AlgorithmLookup` that returns just-materialised winners alongside the on-disk catalog, so a larger winner can use a smaller winner from this same phase as a sub-product. Spot-check the result against the predicted rank, verify, write to disk.

Theorem 1 guarantees that the rank propagated by Phase 1 equals the rank produced by Phase 2 for every entry that materialises successfully (the rare failures are infrastructural – e.g. a sub-product that became unavailable between rounds).

The impact DAG, not a full sweep. Shapes and their derivation candidates form a directed acyclic graph of *impacts*: an edge $\sigma \rightarrow \tau$ exists whenever a rank improvement at shape σ can lower the predicted rank at τ — because τ Kron-, concat- or recombines a copy of σ (an *upward* edge), or because τ is a projection of σ (a *downward* edge; Section 5.4). A win therefore cannot change any shape that is not downstream of it. The search exploits exactly this: the frontier driver (`FrontierClosure`) keeps a priority queue of seeds keyed by ω (best first), pops a seed, and on each improvement pushes back *only* the shapes that seed can impact — so it rides the DAG, touching a shape only when one of its sub-shapes has just improved. We do *not* perform a full Cartesian sweep of all shapes against all strategies every round: work is proportional to the impactful edges actually traversed, not to $|\text{shapes}| \times |\text{strategies}| \times |\text{rounds}|$. The two cost tiers of this DAG also differ — an upward edge is a cheap rank-arithmetic re-evaluation, whereas a downward (projection) edge requires materialising and DCE-ing the parent — so the closure fans out upward eagerly and schedules the downward work.

9.2 Why the split matters

Three reasons.

Throughput. Materialisation involves Java array allocation, factor-matrix arithmetic, spot-check sampling, and disk I/O. A target shape at $\langle 17, 17, 17 \rangle$ takes a fraction of a second to predict and a few seconds to materialise; at $\langle 32, 32, 32 \rangle$ the spread becomes minutes. Postponing materialisation lets the search round terminate as soon as no new predictions are found, even when 50 shapes have pending materialise work.

Correctness of intra-round propagation. A previous incarnation of the closure loop wrote each materialised winner to disk and relied on the next round to reload it. The catalog `FieldAwareLookup` was loaded once at the start of the sweep, so the next round did *not* see the new entry: round $N + 1$ used the same catalog snapshot as round N . The pending overlay fixes this: subsequent rounds see prior-round wins immediately, regardless of whether the JVM was restarted.

Decoupling verification. Verification is a third phase that can run on demand against any materialised scheme – e.g. as a periodic catalog-wide sweep or as a follow-up to a search batch. There is no need to verify every newly-materialised scheme inside the closure loop, only those whose predicted-vs-actual rank match proves the materialise step succeeded (which is the cheap check; true random-matmul spot-check can run in batch later).

9.3 Rebuilding the catalog from atoms: why the closure is multipass

It helps to state the closure’s job in the strongest form: given only the *atoms* (Section 4) — the handful of primitive schemes we do not derive — reconstruct the entire catalog by repeatedly applying the derivation operators of Section 5. The round-based fixed point is exactly what makes this safe and easy: rather than hand-ordering which shape depends on which, each round re-applies every operator to every shape and the loop stops only when nothing improves.

For an *upward*-only operator set this is almost an accounting detail. Kronecker, axis concatenation, recombination and the serendipitous product (Section 5.2.1) all build a *larger* shape from *smaller* ones, so a single sweep in increasing maximum dimension would already suffice: every sub-product a shape needs is final before the shape is processed.

The multipass design earns its keep once an operator runs *downward*. The canonical example is *projection*: a scheme for $\langle n, m, p \rangle$ is obtained from a *larger* one for, say, $\langle n+1, m, p \rangle$ by zeroing one input slice, discarding the matching output slice, and dead-code-eliminating the products that fed only the dropped index. All three axes project. The two outer axes (n, p) each touch one input operand and the output C , so projecting them shrinks C (a row or a column). The inner/contracted axis m is different: it indexes A ’s columns *and* B ’s rows but *not* C , so projecting m removes a term from the contraction sum — C keeps its shape, and the dead-code test consults *both* input factors rather than one input and the output. A cube-to-cube step such as $\langle 26, 26, 26 \rangle \rightarrow \langle 25, 25, 25 \rangle$ projects one index on each of the three axes at once (the index choice per axis is what FMM’s $[[1, 15], [15]]$ bracket records). Two facts make this concrete:

- $\langle 3, 3, 3 \rangle : 25$ is the projection of $\langle 4, 4, 4 \rangle : 49$: pad $\langle 3, 3, 3 \rangle$ with a zero row and column to $\langle 4, 4, 4 \rangle$, apply Strassen recursively, and eliminate the padding — precisely the worked example of [3]. Projection is thus the same operation as padding-plus-DCE, run in reverse.
- At scale, $\langle 25, 25, 25 \rangle : 8359$ is the projection of $\langle 26, 26, 26 \rangle : 8658$; FMM [20] builds most of its non-cube and odd-cube entries this way, projecting a strong even cube onto its neighbours.

Which index is dropped matters: each index has a different set of products localised to it, so dropping the most-localised one eliminates the most and gives the lowest rank. A concrete case — projecting Laderman’s $\langle 3, 3, 3 \rangle : 23$ down one axis to $\langle 2, 3, 3 \rangle$:

dropped index	products eliminated (DCE)	resulting rank
0	5	18
1	7	16
2	7	16

Same scheme, same target shape, but a two-multiplication swing from the choice alone: Laderman concentrates more products on indices 1 and 2 than on index 0, so dropping either of those

eliminates 7 products versus only 5 for index 0. A “drop-the-last-index” rule would have left rank 18 where 16 was available. (The format’s true optimum, $R(\langle 2, 3, 3 \rangle) = 15$, comes from a different scheme, not a projection of Laderman — a reminder that projection’s payoff is bounded by its parent, which is why the closure keeps several parents per format.)

FMM records this choice explicitly (e.g. the bracket in “projection $[[1, 15], [15]]$ ” selects index 15, not the last index), and the operator must try every index per axis rather than only the tail. The best choice is moreover joint in the source scheme and the dropped index — a different parent for the same shape concentrates products differently — so projection, like the serendipitous product, rewards keeping structurally diverse schemes per format rather than only the minimum-rank one. With our recursively-composed schemes the per-index localisation is not predictable in closed form, so we simply enumerate all drop positions, project, and verify — each position is an $O(r)$ support scan plus dead-code elimination, so the exhaustive choice is cheap.

A downward operator destroys the acyclicity in dimension: improving $\langle 26, 26, 26 \rangle$ can lower $\langle 25, 25, 25 \rangle$, which a smaller-first sweep has already finalised. So the catalog cannot be rebuilt in one dimension-ordered pass — it must iterate to a fixed point, re-projecting neighbours after each cube improvement. This is also the *safer* design: instead of special-casing “did some larger shape just improve?”, the fixed point simply re-runs all operators each round and a projection win is absorbed on the next round automatically. Cube quality therefore propagates downward through the whole catalog — a single better $\langle n, n, n \rangle$ improves every nearby rectangular and odd-cube shape at once.

The downward (projection) operator and its *projection-margin* score μ — the criterion that decides when a higher-rank parent projects to a lower-rank child — are defined alongside the derivation strategies in Section 5.5. Operationally the closure fires a bounded *downward wave* on each catalog update: a freshly arrived scheme with $\mu > 0$ is projected into its dimension-neighbours, and any neighbour that improves is enqueued for its own projections; termination is immediate since dimension strictly decreases.

9.4 Pool, allocation enumeration, and pruning

The derivation search at a shape $\langle n, m, p \rangle$ enumerates, over a pool of outer schemes:

- per-axis allocations of $n/m/p$ into the outer scheme’s block counts, optionally filtered by a maximum per-allocation imbalance $\max(\mathbf{a}) - \min(\mathbf{a}) \leq \text{maxImbalance}$ and capped at `maxCombinations` via a top- K -by-balance pre-selector;
- over-allocation by per-axis padding δ up to `maxPadding`, paired with a corresponding tail-peel pattern (Section 5.6); and
- axis-flip orbit masks of the outer scheme, evaluated analytically via per-product block-support bitmasks (`AnalyticalMaskSearch`) so the 8-mask coverage costs only a constant factor over single-mask evaluation.

The default pool (`PoolConfig.simple`) carries a small number of high-value outer schemes — Strassen, Winograd, Smirnov 3-row templates, and a few imported Pan / AT / FMM bases. An `extendedPool` mode adds every `Leaf`-tagged catalog entry as an outer template; that pool is large enough that the `maxCombinations`-capped allocation enumerator becomes load-bearing.

9.5 Allocation optimisation by branch-and-bound

Enumerating allocations naively is hopeless: for a fixed outer scheme of shape $\langle n_o, m_o, p_o \rangle$, the number of ways to split the target is $\binom{N-1}{n_o-1} \binom{M-1}{m_o-1} \binom{P-1}{p_o-1}$, which blows up well before $N = 32$. Yet the inner problem — given a target, template, and orientation, find the allocation minimising $\sum_{\sigma} R(\sigma)$ over the sub-problem multiset (Section 5.3) — is solved *exactly* by `AllocationOptimizer`, a branch-and-bound over the allocation grid.

The search fixes the three axis partitions in turn ($\mathbf{a}_N$, then $\mathbf{a}_M$, then $\mathbf{a}_P$), depth-first. Two ingredients make it cheap:

- **A strong incumbent.** The Kronecker (uniform-allocation) rank is a ready upper bound, seeded before the descent; most subtrees then fall to the bound immediately.
- **An admissible lower bound.** At a partial node — some axes resolved, others open — each product is charged the smallest rank it could still take (the resolved axes’ sizes, a floor on the open ones). Because R is monotone in the dimensions this never overestimates, so a node whose bound already meets the incumbent is pruned without expansion.

Branch-and-bound returns the *same* optimum as the exhaustive grid while visiting far fewer allocations (the result reports `nodes` visited against `fullSpace`).

Budget and what “optimal” means here. The optimiser runs under a `SearchBudget`: `EXACT` (no cap), `upTo`(a known upper bound, e.g. the Kronecker rank), or a `maxNodes` anytime cap. The result carries an `exhaustive` flag, and we propagate it honestly (cf. the optimality discipline of Section 1). When `exhaustive` is true the returned allocation is the *proven global minimum over allocations* for that template and orientation — *optimal-within-scope*, not the tensor rank $R(\langle N, M, P \rangle)$, which a different template or a non-recombination construction may still beat. When the node budget is hit, `exhaustive` is false and the rank is only a best-found *upper bound*. The outer loop (pool of templates $\times$ axis-flip orbit) and the frontier closure wrap this exact inner solve, and are themselves bounded; the catalog never reports an allocation result as a global rank optimum.

9.6 Sound pruning from cheap candidates

The derivation search has three families of candidate strategies at every target shape: multi-base recombination (heavy — iterates the entire pool, every allocation, every axis-flip mask), *concat-split* (cheap — a single axis split, two SOTA lookups), and *Kronecker* (cheap — nested factor enumeration of n, m, p , all SOTA lookups). Cheap candidates finish in milliseconds; the heavy recombination sweep can take minutes on a single shape at high dim.

We exploit this asymmetry. Concat and Kronecker run *first*, producing an upper bound $\hat{r}_{\text{cheap}} = \min(r_{\text{concat}}, r_{\text{kron}})$. The recombination sweep is then seeded with `bestRank` := $\hat{r}_{\text{cheap}}$. Two sound prunes follow:

- *Per-base lower bound.* For an outer base of rank r , the recombination produces r sub-products, each costing ≥ 1 ; the pair-fusion pass (Section 5.7) only applies pairs whose savings are positive, so pair-cost is at least the unpaired sum on degenerate-shape sub-products. Hence total $\geq r$ under any allocation. If $r \geq \text{bestRank}$, the entire base is skipped.
- *Best-so-far gate.* After each (`allocation`, `mask`) tuple completes, the predicted rank is compared against the running `bestRank`, updating the candidate only on strict improvement.

We deliberately do *not* prune *inside* the sub-product accumulation loop. A running sum above `bestRank` does not imply the paired total is above it, since pair-fusion can reduce the sum by tens of percent on cyclic-shape multisets. Adding a slack constant would be a heuristic, and we keep the search at SAT-style soundness: every prune is provably correct.

A canonical case where this matters: at $\langle 12, 12, 12 \rangle$, the Kronecker pair $\langle 2, 4, 4 \rangle = 26 \otimes \langle 6, 3, 3 \rangle = 40$ gives 1040 in milliseconds. Without the reorder, the recombination sweep grinds for several minutes on the AlphaEvolve $\langle 5, 5, 5 \rangle = 93$ base, finding nothing better than 1071, before the final min over all strategies selects Kronecker anyway. With the reorder, AE- $\langle 5, 5, 5 \rangle$ still runs (its trivial lower bound $r = 93 < 1040$), but every allocation that doesn’t beat 1040 is silently dropped instead of being recorded as a (worthless) intermediate best.

9.7 Low- ω contamination

A complementary optimisation reshapes the *order* in which shapes are processed during a closure round. The naive iteration is lexicographic over (n, m, p) , so the search reaches $\langle 12, 12, 12 \rangle$ before it reaches $\langle 17, 17, 17 \rangle$ and so on. But the value of an early win depends on its implied ω : a low- ω seed contaminates every larger shape that consumes it as a sub-product.

We rank candidate outer bases (and known catalog shapes) by their implied exponent $\omega(\langle n, m, p \rangle : r) = \log_n(r) \cdot 3 / (1 + \log_n(m) + \log_n(p))$ under the convention of Section 3, and prioritise:

1. Low- ω seeds first: e.g. $\langle 4, 4, 4 \rangle = 48$ ($\omega \approx 2.7925$; AlphaEvolve over $\mathbb{C}$, the DPS rationalisation [4] over $\mathbb{Q}/\mathbb{R}$), Smirnov $\langle 3, 3, 6 \rangle = 40$, Pan $\langle 3, 4, 7 \rangle = 63$, then up the ladder.
2. Targets that can be *contaminated* by a low- ω seed (via Kronecker, recombination, or concat) are re-evaluated in the same round once the seed is on the overlay, so a fresh ω reduction propagates without a separate round.
3. The shape order within each round respects this dependency: every shape that consumes a seed via a known composition rule waits for the seed’s seed-round completion before its own search starts.

Concretely: when AE- $\langle 4, 4, 4 \rangle = 48$ lands as a seed in round k , every Kronecker target whose factorisation includes $(4, 4, 4)$ on at least one axis pair — $\langle 8, 8, 8 \rangle$, $\langle 12, 12, 12 \rangle$, $\langle 16, 16, 16 \rangle$, and so on — sees its predicted rank drop in round $k + 1$. The “contamination” metaphor captures the spreading: a low- ω reduction at $\langle 4, 4, 4 \rangle$ silently rewrites everyone’s upper bound.

The selection of which schemes act as seeds is encoded in `docs/bases-by-omega.md`, an automatically-regenerated table ranking every pool-eligible base by its raw ω , its ω after imbalance- ≤ 3 restriction, and its ω after axis-flip-canonical and S_3 -canonical orbit reductions. The lowest- ω row at the reference target $\langle 17, 17, 17 \rangle$ is the natural seed for any sweep that targets dim-32-or-higher cubic shapes.

9.8 Pruning is opt-in

A reviewer’s natural question is whether the search uses heuristics to bound the allocation space. We have a strong project-level convention that pruning is opt-in: any heuristic that drops part of the allocation space (`balancedOnly`, `top-N`, “ignore imbalanced”) is a flag the user explicitly turns on, not a silent default. This is a reaction to a recurring failure mode where “the search did not find scheme X” turns out to mean “the search silently pruned the allocation that would have found scheme X.” Unbalanced cubic targets and bridge-shape sub-problems (Hopcroft–Kerr type) live precisely in the region that naive balanced-only pruning discards.

9.9 Caching

Caching exists at two layers. The first is per-scheme *symbolic templates*: for each outer scheme we extract once a tuple of six block-support bitmasks per product (U -row, U -col, V -row, V -col, W -row, W -col), and per-call sub-shape extraction reduces from $O(r \cdot \dim^2)$ to $O(r \cdot \dim)$. The second is a per-(scheme, allocation, peel) cache of the resulting sub-shape multiset; this is a hashing-overhead trade and is currently capped at 200k entries per scheme. A symbolic-template-at-allocation-pattern layer would generalise the second cache; the current implementation captures the common cases without paying the templating cost.

9.10 Progress reporting

A frontier closure over the entire catalog up to dim 32 takes hours of wall-clock. Per CLAUDE.md “Long-running procedures” we emit periodic progress lines at three nested levels: per-round

(round counter + overlay size), per-shape (search/materialise phase markers with timing), and intra-search (current pool entry, current allocation, allocations explored, throughput, best so far). The intra-search level is the one that catches a stuck inner-loop call, which otherwise stays silent for tens of minutes during heavy axis-flip mask enumeration at high dim.

10 Comparison tables

We refresh the comparison tables of Drevet–Islam–Schost 2009 (“DIS09” [3]; Tables 3 and 4 in the original numbering) with the following changes relative to that reference.

- Per field, not per “algebra”. Each of $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, $\mathbb{F}_2$ gets its own column — the fields are kept *distinct* (there is no “ $\mathbb{R}/\mathbb{Q}/\mathbb{Z}$ ” merge) — and ternary-integer ($\mathbb{ZT}$) is surfaced as a per-scheme flag. The inclusion $\mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$ is used only when computing *which* fields a scheme satisfies (field widening), never to collapse columns.
- Commutative axis split out. DIS09 Table 4 (commutative) is preserved as a separate set of columns alongside the non-commutative ones, so the reader cannot accidentally compare Rosowski 21 to Laderman 23 (Section 3).
- Every cell carries an `attribution_for_rank`, which is the earliest source that established the bound. The contemporary importing source (FMM-Lille, Perminov, AlphaTensor bulk imports) is recorded separately and does not appear in the table cells.
- Border-rank entries are split out into a separate appendix table; the main tables report tensor rank only.

10.1 SOTA definition

Throughout the tables, “SOTA” means the lowest known rank for $(\langle n, m, p \rangle, K, \text{commutative})$ where the rank is witnessed by either an explicit catalog entry, a cited bound, or a derived bound. This is a working definition; several corner cases require explicit decisions, currently captured in `paper/sota-conventions.md`:

Mixed-field. When a scheme is valid over $\mathbb{R}$ but its coefficients all lie in $\mathbb{Z}$, we report it as both an R-cell and a Q-cell. The field-widening sweep (`FieldWideningSweep`) drives the cross-population.

Border vs exact. Border-rank bounds are excluded from the tensor-rank tables and appear in the border-rank appendix only.

Partially verified. Cited bounds that lack a verified scheme are flagged with a dagger ($\dagger$) in the cell.

$\mathbb{F}_2$. Integer schemes that mod-2-reduce to valid $\mathbb{F}_2$ schemes are not yet automatically promoted into the $\mathbb{F}_2$ column. Section 11.

10.2 Non-commutative ranks per field

The four sub-tables below are scaffolds; their cells are populated by the regeneration tooling (separate roadmap item) once SOTA conventions are finalised.

10.3 Commutative ranks per field

(Commutative results over $\mathbb{C}$ and $\mathbb{F}_2$ are sparse in the literature; we list them only when at least one entry exists.)

Table 6: Non-commutative rank $\mathbb{Q}\langle n, m, p \rangle : r$ for selected shapes. Cells show **rank attribution-for-rank**. † marks a cited bound whose explicit scheme is not in the catalog.

Shape	SOTA rank	Attribution (earliest known)	Year
$\langle 2, 2, 2 \rangle$	7	Strassen	1969
$\langle 2, 2, 3 \rangle$	11	Hopcroft–Kerr	1971
$\langle 2, 3, 3 \rangle$	15	Hopcroft–Kerr	1971
$\langle 2, 3, 4 \rangle$	20	Smirnov	2013
$\langle 3, 3, 3 \rangle$	23	Laderman	1976
$\langle 3, 3, 4 \rangle$	29	Smirnov	2013
$\langle 3, 3, 6 \rangle$	40	Smirnov	2017
$\langle 4, 4, 4 \rangle$	49	Strassen recurs.	1969
$\langle 4, 4, 5 \rangle$	58	Smirnov	2017
$\langle 5, 5, 5 \rangle$	93†	Perminov (Z-ext)	2024
$\langle 7, 7, 7 \rangle$	250	Sedoglavic–Smirnov	2017
$\langle 9, 9, 9 \rangle$	511	catalog composition	2026
$\langle 17, 17, 17 \rangle$	2930†	Schwartz–Zwecher	2025
$\langle 32, 32, 32 \rangle$	TBD	—	—

Table 7: Non-commutative rank $\mathbb{R}\langle n, m, p \rangle : r$ for shapes where R differs from Q, or where R is the canonical reference. Otherwise the value equals Table 6.

Shape	SOTA rank	Notes	Year
$\langle 2, 2, 2 \rangle$	7	Strassen (R coincides with Q)	1969
$\langle 3, 3, 3 \rangle$	23	Laderman	1976
$\langle 4, 4, 4 \rangle$	49	Strassen ² ; R-only beat over Q is 48 in $\mathbb{C}$	1969
$\langle 4, 4, 5 \rangle$	58	Smirnov	2017
$\langle 5, 5, 5 \rangle$	100	catalog composition (R)	2026

10.4 Frontier visualisation

A companion figure – regenerated from the catalog – plots the SOTA frontier as the relationship between shape complexity $\sqrt[3]{nmp}$ and ω -implied lower bound $\log_{\sqrt[3]{nmp}}(r) \cdot 3$. The plot is split per field and per commutativity tag and lives in `paper/figures/frontier.pdf`; see Section 11 for the tooling pipeline that emits it.

11 Open questions and future work

We close with what the catalog cannot currently answer and where the next bits of attention should go.

Disjoint-sum search. The frontier-closure search of Section 9 composes via Kronecker, axis concat, recombination, peel, and pair fusion. It does *not* construct τ -theorem disjoint sums; those are imported from external sources (Schwartz–Zwecher 2025; FMM-Lille). A constructive disjoint-sum operator would close several gaps where our search currently predicts 2940 at $\mathbb{Q}\langle 17, 17, 17 \rangle$ while SZ 2025 produces 2930 directly. The operator’s feasibility check is non-trivial – “does this tensor admit a disjoint sum into these legs?” – and warrants a dedicated note.

Salt-tolerant prediction. Section 6 flagged that any post-construction sharing discovery (random sign salting, column fusion, coefficient mixing) would break the search-rank-equals-

Table 8: Non-commutative rank $\mathbb{C}\langle n, m, p \rangle : r$ for shapes where C strictly improves on R / Q.

Shape	SOTA rank	Notes	Year
$\langle 4, 4, 4 \rangle$	48	AlphaEvolve	2024
$\langle 5, 5, 5 \rangle$	96	AlphaEvolve	2024

Table 9: Non-commutative rank $\mathbb{F}_2\langle n, m, p \rangle : r$ for shapes where $\mathbb{F}_2$ strictly improves on characteristic-zero results, or where the $\mathbb{F}_2$ result is the canonical reference.

Shape	SOTA rank	Attribution	Year
$\langle 4, 4, 4 \rangle$	47	AlphaTensor	2022
$\langle 4, 5, 5 \rangle$	76	AlphaTensor	2022
$\langle 5, 5, 5 \rangle$	96	AlphaTensor	2022

materialised-rank invariant. We believe the right shape for adding such salt is as a separate “polish” phase after the closure converges, but the interaction with the lazy materialise design needs to be worked out before implementing.

$\mathbb{F}_2$ promotion from $\mathbb{Z}$. Every integer scheme reduces mod 2 to give an $\mathbb{F}_2$ scheme; some of these reductions are strictly tighter than the rank claimed over $\mathbb{Z}$ because coefficient cancellations land. The catalog does not yet promote integer schemes into the $\mathbb{F}_2$ column automatically; doing so would likely surface a number of “free” $\mathbb{F}_2$ entries below the current AlphaTensor frontier.

Border-rank track. The current catalog is exact-rank only. A border-rank track would let us include results like Bini’s $\mathbb{R}\langle 2, 2, 2 \rangle$ border rank 7 constructions and Schönhage’s τ -theorem upper bounds. A separate `border-bounds.json` exists and is used by the SPA but is not yet integrated into the comparison tables.

Lower-bound integration. `lower-bounds.json` carries the best known lower bounds per shape – Strassen’s $\mathbb{R}\langle 2, 2, 2 \rangle \geq 7$, Bläser’s family, the recent Wang 2026 $\mathbb{F}_2\langle 3, 3, 3 \rangle \geq 20$. Adding a “gap-to-lower-bound” column to each comparison table would make the SOTA frontier read more cleanly: a cell saying “49 (R, gap 4 to lower-bound 45)” tells more than “49 (R)”.

Border between catalog and hand-crafted-discovery. The non-overlap property of Section 6 draws a sharp line we want to defend more carefully. Concretely: when a hand-crafted construction lands at a shape where composition was *also* predicting the same rank, the lineage should record both discoveries and the comparison table should report both. Today the catalog picks one. A two-attribution column would force the distinction into the table.

Reproducibility tooling. The numeric tables in Section 10 are populated by hand on first pass; a regeneration pipeline (separate roadmap item) takes `docs/catalog.json`, `docs/cited-bounds.json`, and `docs/derived-bounds.json` and emits `paper/tables/*.tex` per the conventions of Section 10.1. The bigger work item is finalising the SOTA conventions document (`paper/sota-conventions.md`) so the regeneration pipeline has a deterministic rule for every corner case.

12 On the role of the AI coding agent

This catalog, its search engine, and much of this paper were developed in close collaboration with an AI coding agent (Anthropic’s Claude, via Claude Code). We record the division of labour

Table 10: Commutative rank $\mathbb{Q}\langle n, m, p \rangle^c$ for selected shapes. These bounds are valid for scalar matrix multiplication only; they do NOT lift to recursive matmul over non-commutative rings.

Shape	SOTA rank	Attribution	Year
$\langle 2, 2, 2 \rangle$	6	Waksman / Hopcroft–Kerr	1970 / 1971
$\langle 3, 3, 3 \rangle$	21	Rosowski Thm 2/3 / Makarov	1986 / 2010
$\langle 4, 4, 4 \rangle$	48	Waksman	1970
$\langle 5, 5, 5 \rangle$	TBD	—	—

Table 11: Commutative rank $\mathbb{R}\langle n, m, p \rangle^c$. Most cells coincide with Table 10.

Shape	SOTA rank	Notes
$\langle 2, 2, 2 \rangle$	6	Waksman, coincides with Q
$\langle 3, 3, 3 \rangle$	21	Rosowski, coincides with Q
$\langle 4, 4, 4 \rangle$	48	Waksman, coincides with Q

explicitly — in the same spirit as the optimality discipline of the rest of the paper: say exactly what was done, and how confidently it can be attributed.

A fairly clean split emerged between *direction* and *execution*. The human author supplied the direction: which questions to pose, which perspectives were worth digging into, what counted as an interesting result, plus domain framing and final judgement. The agent supplied most of the execution — implementation, routine derivations, test scaffolding, provenance bookkeeping — and a substantial share of the *theory*, but only *once a direction was fixed*. Two concrete cases, with our (necessarily subjective) attribution:

- **The recombination branch-and-bound** — the allocation/orbit search that assigns target dimensions to a base’s slots and prunes by a rank bound — was roughly **95% agent**: both the idea to cast allocation as a bounded search and its implementation.
- **The exhaustive enumeration of recombination multisets** (Section 8): the *theory* — the per-axis decoupling lemma that turns an intractable $39k^7$ search into an exact, direction-stratified enumeration — and its implementation were again roughly **95% agent**. But the *idea to investigate the multiset perspective at all* was **100% the human author**: the agent did not propose that this was a fruitful lens, it only developed and executed it once asked.

A second, less flattering pattern was just as consistent: the agent **hallucinated frequently**. It routinely produced confident, plausible, and *wrong* intermediate claims — false “facts”, derivations that did not hold, and bugs reported as fixes. A representative case occurs in this very work: the agent first concluded that the $\langle 2, 2, 2 \rangle$ recombination multisets collapse to *two* classes once per-axis block-flips are quotiented out — which silently contradicts the paper’s own premise that Strassen and Winograd differ. The error surfaced only when the claim was cross-checked against the catalog’s independent cost machinery, exactly the sort of confident-but-false output that pervaded the collaboration. A second, *recurring* instance was more telling still: asked to explain how the Université de Lille FMM catalog attains a particular finite-size rank, the agent would repeatedly reach for Schönhage’s τ -theorem (the asymptotic sum inequality) [18] as the mechanism — something this project neither runs nor derives, and which does not produce the concrete finite schemes in question (those come from FMM’s own recursive concatenation and projection). This confabulation was strikingly persistent: it recurred across sessions and survived explicit standing instructions recorded in the agent’s persistent project memory that warn against exactly it, requiring the human to steer the agent off it hard and often. That written, recalled-in-context rules did not reliably suppress a favoured hallucination is itself worth

recording. (Fittingly, an earlier draft of this very paragraph mislocated those instructions in the repository’s `CLAUDE.md` — they live in the agent’s memory files — an error caught only by the same grep-and-verify gating the paragraph prescribes.) The practical consequence is structural: *every* agent-produced claim in this work had to be gated behind an independent check — a verifier, an exhaustive recomputation, or an adversarial second pass — and the project’s emphasis on cheap, automatic verification (Section 6) is in part a direct response to how unreliable the agent’s unverified assertions were. Hallucination was the norm, not the exception; what made the collaboration productive was not trusting the agent but *cheaply catching* it.

We find the split informative. The agent is strongest at turning a well-posed direction into correct theory and code, and at the breadth a research-grade catalog demands (hundreds of schemes, fields, provenance, verification). The scarce, human-supplied ingredient is the choice of question. The multiset case is the sharp illustration: once “look at the multiset, and list them exhaustively” was posed, the decoupling lemma, its completeness argument, the registration of the novel triple-aggregation base, and even this section’s prose followed largely from the agent — yet nothing in the agent’s behaviour generated the framing itself. The branch-and-bound is the complementary case, where both framing and execution were the agent’s, the human contributing review and acceptance.

Some problems defeated the agent outright, and we record them as honest boundaries rather than omit them. The clearest is the Hopcroft–Kerr 1971 construction for $\langle 2, m, p \rangle$: the agent could not rebuild a constructive derivation of the published ranks with our own toolkit. (The related *below*-HK schemes — those beating the HK ranks, such as AlphaEvolve’s $\langle 2, 4, 5 \rangle$ at 32 — need a characteristic-0 half-symmetrisation $\frac{1}{2}(a_i \pm a_j)$ our integer toolkit provably cannot express; the HK ranks themselves are attained by ternary integer schemes, so no such excuse applies there.) The author pursued the same question in parallel with a *different* agent (OpenAI’s ChatGPT 5.5), whose style was markedly more theoretical than Claude Code’s implementation-first one — arguing about the underlying algebra and attempting proofs through Mathematica scripts rather than enumerative code. That line was equally inconclusive: it neither produced a derivation within reach of our methods nor established that the $\langle 2, m, p \rangle$ case is unreachable, nor did it surface an error in the original paper. The HK schemes therefore stand in the catalog as *imported but not re-derived*: explicit factor matrices (via the FMM-Lille catalog), machine-verified and correctly attributed, but with no in-toolkit constructive route to them — consistent with our rule of importing what we cannot yet derive, and a reminder that two independent agents, with opposite working styles, can both stall on the same genuinely hard point.

Tooling. The bulk of the work was carried out with Anthropic’s Claude Code on Claude Opus 4.7, with a smaller share on Opus 4.8; the most recent work (including parts of this section) runs on its newly released successor, Claude Fable 5. The hallucination rate noted above is for the Opus 4.x models specifically, and is a moving target — a reader reproducing this with a later model should not assume the same reliability, in either direction.

These percentages are self-assessments, not measurements, and are offered in the same honest-bounds spirit as the rest of the paper — an upper bound on the author’s share of the keystrokes, rather than a proven division of credit. We state them because reproducibility increasingly includes *how* a result was produced, and because the boundary this work happened to draw — human chooses the lens, agent builds the optics — may be a useful data point for others doing mathematics with these tools.

References

- [1] Vladimir P. Burichenko. On symmetries of the Strassen algorithm. *arXiv:1408.6273*, 2014.

- [2] Hans F. de Groote. On varieties of optimal algorithms for the computation of bilinear mappings. ii. optimal algorithms for 2×2 -matrix multiplication. *Theoretical Computer Science*, 7(2):127–148, 1978.
- [3] Charles-Éric Drevet, Md. Nazrul Islam, and Éric Schost. Optimization techniques for small matrix multiplication. *Theoretical Computer Science*, 412(22):2219–2236, 2011. Preprint dated 2009; published 2011.
- [4] Jean-Guillaume Dumas, Clément Pernet, and Alexandre Sedoglavic. A non-commutative algorithm for multiplying 4×4 matrices using 48 non-complex multiplications, 2025. arXiv:2506.13242.
- [5] Alhussein Fawzi et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53, 2022.
- [6] John E. Hopcroft and Leslie R. Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM Journal on Applied Mathematics*, 20(1):30–36, 1971.
- [7] Md. Anwarul Islam. Bilinear algorithms for matrix multiplication and their applications. *PhD thesis, University of Western Ontario*, 2009.
- [8] Manuel Kauers, Jakob Moosbauer, and Isaac Wood. Exploiting the structure in tensor decompositions for matrix multiplication. *arXiv preprint arXiv:2602.11041*, 2026.
- [9] Julian D. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bulletin of the AMS*, 82(1):126–128, 1976.
- [10] O. M. Makarov. An algorithm for multiplication of 3×3 matrices. *Zh. Vychisl. Mat. i Mat. Fiz.*, 26(2):293–294, 320, 1986.
- [11] Alexander Novikov et al. Alphaevolve: a coding agent for scientific and algorithmic discovery, 2025. Google DeepMind. arXiv:2506.13131.
- [12] V. Ya. Pan. Strassen’s algorithm is not optimal – trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations. In *Proc. 19th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 166–176, 1978.
- [13] V. Ya. Pan. New fast algorithms for matrix operations. *SIAM Journal on Computing*, 9(2):321–342, 1980.
- [14] Andrew I. Perminov. Fastmatrixmultiplication. <https://github.com/dronperminov/FastMatrixMultiplication>. Accessed 2026-06.
- [15] Andrew I. Perminov. Fast matrix multiplication via ternary meta flip graphs. *arXiv preprint arXiv:2511.20317*, 2025.
- [16] Andrew I. Perminov. Fast matrix multiplication in small formats: Discovering new schemes with an open-source flip graph framework. *arXiv preprint arXiv:2603.02398*, 2026.
- [17] Andreas Rosowski. Fast commutative matrix algorithms. *arXiv preprint arXiv:1904.07683*, 2019.
- [18] Arnold Schönhage. Partial and total matrix multiplication. *SIAM Journal on Computing*, 10(3):434–455, 1981.
- [19] Oded Schwartz and Eyal Zwecher. Disjoint-sum constructions for fast matrix multiplication. *hal-05121550, preprint*, 2025.

- [20] Alexandre Sedoglavic et al. Fmm: Fast matrix multiplication catalog. <https://fmm.univ-lille.fr/>. Accessed 2026-06.
- [21] Alexey Vladimirovich Smirnov. *Bilinear Complexity and Practical Algorithms for Matrix Multiplication*. PhD thesis, Computing Center of the Russian Academy of Sciences, 1986.
- [22] Alexey Vladimirovich Smirnov. Bilinear complexity and practical algorithms for matrix multiplication. *Computational Mathematics and Mathematical Physics*, 53(12):1781–1795, 2013.
- [23] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.
- [24] Abraham Waksman. On winograd’s algorithm for inner products. *IEEE Transactions on Computers*, C-19(4):360–361, 1970.